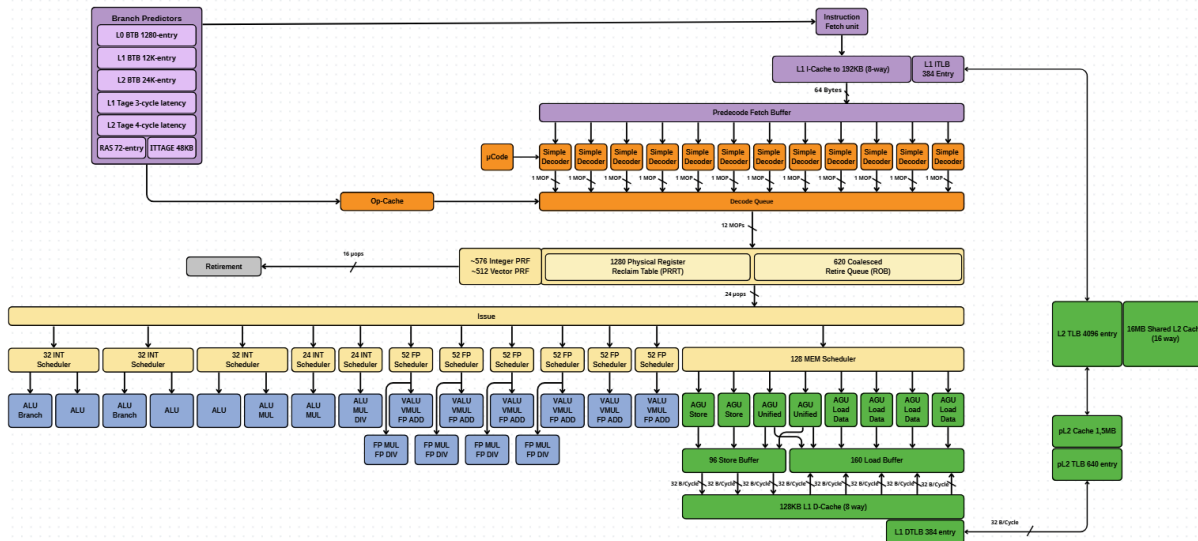


P core

P Core



Báo cáo kỹ thuật giới thiệu kiến trúc P Core
trong sơ đồ đã cho

1. Mở đầu

Sơ đồ “P Core” được cung cấp mô tả một nhân xử lý hiệu năng cao, được tổ chức theo phong cách vi kiến trúc hiện đại với pipeline rộng, khả năng thực thi ngoài thứ tự sâu, hệ thống dự đoán nhánh nhiều tầng, và một backend rất mạnh cho cả tính toán số nguyên, dấu chấm động, vector và truy cập bộ nhớ. Theo ngữ cảnh của bài toán, đây được xem là một **CPU ARM**; tuy nhiên, bản thân sơ đồ chủ yếu cho thấy **cấu trúc vi kiến trúc của nhân xử lý** hơn là các chi tiết trực tiếp của tập lệnh. Vì vậy, mục tiêu của báo cáo này không phải là xác nhận một sản phẩm thương mại cụ thể, mà là **giải thích rõ ràng cách core này được tổ chức, dữ liệu và lệnh đi qua nó như thế nào, từng khối có vai trò gì, và vì sao những con số trong sơ đồ lại cho thấy đây là một nhân hiệu năng cao.**

Báo cáo được viết theo phong cách giới thiệu kỹ thuật dành cho người đọc muốn hiểu kiến trúc của core từ mức khái quát đến mức chi tiết. Thay vì chỉ liệt kê thông số, bài viết sẽ đi theo logic vận hành của core: lệnh được dự đoán và nạp ra sao, cách giải mã và đổi tên thanh ghi hoạt động như thế nào, các cụm thực thi được chia tổ chức ra sao, hệ thống bộ nhớ hỗ trợ việc tải và lưu dữ liệu như thế nào, và toàn bộ các thành phần này phối hợp để tạo nên một nhân xử lý có năng lực tính toán lớn như thế nào.

2. Cái nhìn tổng thể về P Core

Nhìn ở mức toàn cục, đây là một **performance core**, nghĩa là một nhân được thiết kế với ưu tiên hàng đầu là **tăng hiệu năng xử lý**, đặc biệt là hiệu năng đơn luồng và khả năng duy trì throughput cao khi chạy các chương trình phức tạp. Khác với các nhân tiết kiệm điện thường có pipeline hẹp và tài nguyên ít hơn, sơ đồ này cho thấy một thiết kế đầu tư mạnh ở gần như mọi lớp của pipeline.

Ngay từ front-end, hệ thống đã có một **bộ dự đoán nhánh rất lớn và nhiều tầng**, cho thấy mục tiêu là giữ dòng lệnh luôn liên tục, giảm các lần rẽ nhánh sai. Sau đó, phần nạp lệnh có **L1 instruction cache lớn**, băng thông fetch rộng, và một **cụm giải mã 12-wide**, tức có thể chuyển rất nhiều lệnh thành thao tác nội bộ chỉ trong một chu kỳ nếu điều kiện thuận lợi. Đi tiếp xuống backend, core có **khả năng issue 24 μops, retire 16 μops**, và duy trì một số lượng rất lớn lệnh đang bay nhờ **ROB 620 entry** cùng hàng loạt tài nguyên đổi tên thanh ghi và bảng quản lý thanh ghi vật lý. Cuối cùng, hệ thống thực thi được chia thành ba miền chính: **số nguyên, vector/dấu chấm động**, và **bộ nhớ**, với số lượng scheduler, execution units, AGU, load/store buffers và cache rất lớn.

Tất cả các yếu tố đó cho thấy đây không phải một thiết kế chỉ “rộng ở một chỗ”, mà là một thiết kế **cân bằng toàn hệ thống**. Một core chỉ có decode rộng nhưng predictor yếu thì sẽ thường xuyên bị đói lệnh. Một core có scheduler lớn nhưng bộ nhớ yếu thì sẽ nhanh chóng bị chặn bởi cache miss và độ trễ load. Trong sơ đồ này, các thành phần đều được mở rộng đồng bộ, cho thấy rõ triết lý thiết kế: **đã muốn hiệu năng cao thì phải đầu tư toàn pipeline**.

3. Front-end: nơi bắt đầu của hiệu năng

Trong bất kỳ CPU hiện đại nào, front-end đóng vai trò như “hệ thống cấp lệnh” cho toàn bộ backend. Nếu front-end không cung cấp đủ lệnh đúng thời điểm, toàn bộ phần sau dù có mạnh đến đâu cũng không thể phát huy được. Ở P Core này, front-end được thiết kế rất mạnh, và đây là một trong những đặc điểm quan trọng nhất của sơ đồ.

Phần đầu tiên người ta nhìn thấy là cụm **Branch Predictors**. Đây là hệ thống dự đoán nhánh, có nhiệm vụ đoán xem sau một lệnh rẽ nhánh thì luồng lệnh sẽ đi theo hướng nào, địa chỉ lệnh tiếp theo là gì, và nếu chương trình có gọi hàm rồi quay lại thì địa chỉ trả về nằm ở đâu. Core này không dùng một predictor đơn lẻ mà dùng một hệ thống nhiều tầng gồm **L0 BTB 1280-entry, L1 BTB 12K-entry, L2 BTB 24K-entry, L1 TAGE 3-cycle latency, L2 TAGE 4-cycle latency, RAS 72-entry, và ITTAGE 48KB**.

Ý nghĩa của thiết kế này là core muốn đạt đồng thời hai mục tiêu. Mục tiêu thứ nhất là **phản ứng nhanh** với các nhánh nóng, nhánh gần, nhánh lặp lại thường xuyên. Đó là vai trò của các tầng BTB gần hơn và nhanh hơn. Mục tiêu thứ hai là **độ chính xác cao** với các mẫu điều khiển phức tạp hơn, ví dụ các nhánh phụ thuộc vào lịch sử dài, các đoạn mã có cấu trúc điều khiển sâu, hoặc hành vi rẽ nhánh khó dự đoán. Những predictor như TAGE và ITTAGE thường được dùng vì chúng có khả năng học được các tương quan lịch sử dài tốt hơn.

Điểm rất đáng chú ý là kích thước predictor trong sơ đồ này lớn bất thường nếu so với các nhân nhỏ. Việc dành tới **48KB cho ITTAGE** và dùng BTB nhiều tầng với hàng chục nghìn entry cho thấy nhà thiết kế chấp nhận đầu tư mạnh diện tích silic để đổi lấy việc giảm branch misprediction. Trong thực tế, mỗi lần đoán sai nhánh có thể làm pipeline phải xóa bỏ một loạt công việc đã thực hiện, dẫn đến mất nhiều chu kỳ. Một core càng rộng, chi phí của mỗi lần đoán sai càng lớn, bởi vì mỗi chu kỳ có thể đang có rất nhiều lệnh đang được xử lý song song. Do đó, ở một performance core, predictor mạnh gần như là điều kiện bắt buộc.

Bên cạnh predictor là **Instruction Fetch Unit**, có nhiệm vụ lấy lệnh từ bộ nhớ lệnh gần nhất của core. Đơn vị này kết nối với **L1 I-Cache 192KB (8-way)** và **L1 ITLB 384 entry**. Dung lượng **192KB cho L1 I-cache** là một chi tiết rất đáng nói. Một instruction cache lớn giúp giữ được nhiều đoạn mã nóng trong cache hơn, giảm số lần phải đi xa hơn xuống các tầng cache tiếp theo. Điều này có lợi cho những ứng dụng lớn, nhiều hàm, nhiều khối rẽ nhánh, hoặc có footprint mã khá rộng. Với một core muốn duy trì front-end rộng và đều, instruction cache lớn là một lựa chọn rất hợp lý.

Trên sơ đồ còn có đường **64 Bytes** từ L1 I-cache xuống **Predecode Fetch Buffer**. Nghĩa là trong mỗi chu kỳ, về lý thuyết front-end có thể kéo xuống một lượng byte lệnh khá lớn. Điều này tương thích với việc phía sau có tới **12 simple decoders**, bởi nếu băng thông fetch hẹp quá thì dàn decoder rộng cũng không có gì để làm. Có thể hình dung rằng front-end của core này được thiết kế để không chỉ dự đoán đúng, mà còn **đưa được thật nhiều byte lệnh đến khu vực giải mã trong mỗi chu kỳ**.

4. Predecode, decode và op-cache: biến byte lệnh thành thao tác nội bộ

Sau khi byte lệnh được đưa xuống từ I-cache, chúng đi vào **Predecode Fetch Buffer**. Khối này là một lớp đệm trung gian, có nhiệm vụ chuẩn bị instruction stream cho bước decode. Trong nhiều kiến trúc hiện đại, predecode giúp xác định ranh giới lệnh, gom nhóm dữ liệu hợp lý, và làm giảm áp lực thời gian cho các tầng sau. Nói cách khác, nó là bước “tiền xử lý” để các decoder làm việc hiệu quả hơn.

Ngay sau đó là **12 Simple Decoders**. Đây là một con số rất lớn. Mỗi decoder trên sơ đồ được ghi là **1 MOP**, và tổng đầu ra xuống **Decode Queue** là **12 MOPs**. Điều đó cho thấy core có thể giải mã tối đa 12 thao tác mức decode trong một chu kỳ. Đây là một front-end rất rộng. Trong thực tế, khả năng đạt trần này còn phụ thuộc loại lệnh, độ thẳng của dòng lệnh, mức độ thuận lợi của fetch, và việc lệnh có đi qua uCode hay không, nhưng riêng bề rộng lý thuyết thôi cũng đã cho thấy đây là một core nhắm đến IPC cao.

Sơ đồ đồng thời cho thấy sự hiện diện của **uCode**, tức đường vi mã. Điều này rất quan trọng về mặt thiết kế. Không phải mọi lệnh đều đơn giản như nhau. Có những lệnh phức tạp, các thao tác đặc biệt, hoặc các trường hợp hệ thống cần được phân rã thành nhiều bước nhỏ. Thay vì làm dàn decoder thường trở nên quá phức tạp, nhà thiết kế cho phép một số trường hợp đi qua microcode. Đây là cách tổ chức thường gặp trong các vi kiến trúc hiện đại vì nó vừa giữ cho đường decode nhanh với lệnh phổ thông, vừa vẫn hỗ trợ được các thao tác phức tạp.

Một phần rất đáng chú ý khác là **Op-Cache**. Op-cache nằm như một đường rẽ khác dẫn thẳng xuống **Decode Queue**, bỏ qua phần decode truyền thống. Ý nghĩa của op-cache là core có thể lưu lại các thao tác nội bộ đã được giải mã từ trước của những đoạn mã lặp đi lặp lại. Khi chương trình quay lại các đoạn mã đó, front-end không cần giải mã lại từ byte

lệnh nữa mà có thể lấy trực tiếp các thao tác từ op-cache. Cách làm này giúp giảm tải cho decoder, cải thiện độ trễ ở front-end và thường có lợi lớn cho các vòng lặp nóng hoặc các đường thực thi thường xuyên được gọi đến.

Từ góc nhìn của người đang tìm hiểu kiến trúc, có thể xem front-end của core này gồm ba khả năng phối hợp với nhau. Thứ nhất là predictor giúp biết nên nạp lệnh nào. Thứ hai là fetch + I-cache giúp đưa byte lệnh xuống đủ nhanh. Thứ ba là decode + op-cache giúp biến byte lệnh thành thao tác nội bộ với tốc độ rất cao. Khi ba phần này đều mạnh, backend sẽ có nhiều cơ hội được “nuôi đủ lệnh” hơn.

5. Decode queue và giai đoạn chuyển sang thực thi ngoài thứ tự

Sau khi các thao tác được tạo ra từ decoder hoặc op-cache, chúng đi vào **Decode Queue**. Trên sơ đồ, đường từ decode queue đi xuống phần đổi tên và phân bổ tài nguyên được ghi là **12 MOPs**, cho thấy bề rộng của dòng lệnh tiếp tục được duy trì. Decode queue về bản chất là lớp đệm giữa front-end và phần rename/dispatch. Nó giúp hấp thụ sự dao động giữa tốc độ giải mã và tốc độ cấp phát tài nguyên phía sau.

Đây là một chi tiết rất quan trọng trong các core rộng. Trong thực tế, front-end và backend hiếm khi luôn hoạt động ở tốc độ hoàn toàn đồng bộ. Có lúc predictor và fetch rất thuận lợi, decoder có thể đẩy ra nhiều thao tác; nhưng backend lại có thể đang tạm chậm vì ROB gần đầy, hoặc tài nguyên rename tạm căng, hoặc một số cụm thực thi đang bận. Decode queue giúp tách nhịp của hai thế giới đó ra khỏi nhau ở một mức nhất định, nhờ vậy pipeline ổn định hơn.

Kể từ đây, core thực sự bước vào vùng **out-of-order execution**, tức vùng mà lệnh không còn nhất thiết phải được thực hiện đúng theo thứ tự ban đầu nữa, miễn là kết quả cuối cùng vẫn đúng theo mô hình kiến trúc.

6. Rename, PRF, PRRT và ROB: trái tim của thực thi ngoài thứ tự

Một trong những phần nói lên “đẳng cấp” của core này nhiều nhất chính là cụm tài nguyên đổi tên và quản lý trạng thái đang bay. Sơ đồ ghi rõ các tài nguyên gồm **~576 Integer PRF**, **~512 Vector PRF**, **1280 Physical Register Reclaim Table (PRRT)**, và **620 Coalesced Retire Queue (ROB)**.

Để hiểu ý nghĩa của những con số này, cần hình dung cách một core out-of-order hoạt động. Khi chương trình viết ra các thanh ghi kiến trúc, ví dụ như các thanh ghi số nguyên hay vector của ISA, thì ở bên trong CPU, những thanh ghi đó không được ánh xạ cố định vào một vị trí vật lý duy nhất. Thay vào đó, CPU dùng cơ chế **register renaming** để gán cho mỗi phiên bản mới của thanh ghi một thanh ghi vật lý riêng. Nhờ đó, những phụ thuộc giả do cùng tên thanh ghi sẽ được loại bỏ, cho phép các lệnh độc lập thực sự được chạy song song.

Trong sơ đồ này, việc tách riêng **Integer PRF** và **Vector PRF** cho thấy core có hai vùng tài nguyên vật lý lớn dành cho hai loại dữ liệu khác nhau. Điều này có lợi vì khối số nguyên và khối vector/dấu chấm động thường có đặc tính sử dụng khác nhau. Nếu dồn chung tất cả vào một không gian nhỏ hơn, xung đột tài nguyên sẽ dễ xảy ra hơn. Với quy mô khoảng **~576** thanh ghi vật lý số nguyên và **~512** thanh ghi vật lý vector, core có thể giữ được rất nhiều kết quả trung gian mà không phải vội vàng giải phóng.

PRRT 1280 có thể được hiểu như một cơ chế hỗ trợ quản lý và thu hồi thanh ghi vật lý. Trong các bộ xử lý lớn, việc theo dõi thanh ghi nào còn sống, thanh ghi nào có thể thu hồi sau retirement, và thu hồi vào lúc nào là một công việc rất quan trọng. Nếu thu hồi chậm hoặc kém hiệu quả, front-end và rename có thể bị nghẽn vì thiếu thanh ghi vật lý, ngay cả khi các đơn vị thực thi còn rảnh. Kích thước PRRT lớn cho thấy core này đầu tư nhiều vào việc quản lý vòng đời của thanh ghi vật lý.

Quan trọng nhất là **ROB 620 entry**. ROB, hay Reorder Buffer, là nơi giữ thông tin về các lệnh đã được decode, rename và cấp phát tài nguyên nhưng chưa retirement. Đây là thành phần trung tâm của out-of-order execution. Một ROB lớn cho phép core giữ được rất nhiều lệnh “đang bay” cùng lúc. Điều này cực kỳ quan trọng trong hai trường hợp. Trường hợp thứ nhất là khi chương trình có nhiều lệnh độc lập, core cần giữ nhiều lệnh để tìm ra những lệnh có thể thực thi song song. Trường hợp thứ hai là khi chương trình gặp các thao tác có độ trễ dài, nhất là truy cập bộ nhớ. Nếu ROB đủ sâu, core vẫn có thể kéo thêm nhiều lệnh ở phía sau vào hệ thống để tìm những lệnh khác làm tiếp, thay vì đứng yên chờ.

Một ROB 620-entry là dấu hiệu rất rõ ràng của một **core rất lớn và rất tham vọng về hiệu năng**. Những nhân nhỏ, tiết kiệm điện, thường không cần hoặc không đủ khả năng duy trì cửa sổ lớn như vậy. Chỉ khi muốn che giấu nhiều độ trễ, tận dụng predictor mạnh và backend rộng, người ta mới đầu tư vào một ROB sâu như thế.

7. Issue và retirement: nhip vào và nhip ra của backend

Từ khu vực rename/ROB, các thao tác đi xuống **Issue**, với băng rộng được ghi là **24 μops**. Điều này cho thấy trong điều kiện thuận lợi, core có thể phát hành tối đa 24 vi lệnh xuống các scheduler và execution units trong một chu kỳ. Đây là một con số rất lớn. Nó cho thấy backend không chỉ đơn thuần “mạnh”, mà là mạnh ở mức đủ để cần một front-end rất khỏe và một cửa sổ OOO rất sâu để cung cấp nguyên liệu.

Ở phía cuối pipeline là **Retirement**, với băng rộng **16 μops**. Đây là số vi lệnh mà core có thể hoàn tất về mặt kiến trúc trong một chu kỳ. Retirement là bước cuối cùng bảo đảm trạng thái của CPU được cập nhật theo đúng thứ tự chương trình, dù trước đó việc thực thi có diễn ra ngoài thứ tự.

Khi đặt ba con số cạnh nhau, gồm **12 MOPs decode**, **24 μops issue**, và **16 μops retire**, có thể thấy đây là một thiết kế được cân nhắc kỹ. Decode đủ rộng để nuôi backend. Issue rất rộng để tận dụng tối đa cơ hội song song. Retirement cũng đủ mạnh để tránh trạng thái bị “dồn ứ” ở cuối ROB. Nếu retirement quá hẹp so với issue, ROB sẽ sớm đầy lên dù execution units vẫn xử lý tốt. Ngược lại, nếu issue hẹp, một ROB lớn và nhiều execution units cũng khó phát huy. Sơ đồ này cho thấy rõ tư duy cân bằng đó.

8. Cấu trúc scheduler: chia nhỏ để điều phối hiệu quả

Ngay dưới thanh **Issue**, sơ đồ chia backend thành nhiều scheduler khác nhau. Cụ thể có **3 khối 32 INT Scheduler**, **2 khối 24 INT Scheduler**, **6 khối 52 FP Scheduler**, và **1 khối 128 MEM Scheduler**.

Việc chia thành nhiều scheduler phản ánh một nguyên lý thiết kế rất quan trọng. Ở các CPU rộng, một scheduler duy nhất quá lớn sẽ vừa khó chạy nhanh, vừa rất tốn năng lượng và diện tích. Bằng cách phân cụm scheduler theo miền chức năng, nhà thiết kế có thể vừa tăng tổng băng rộng issue, vừa kiểm soát tốt hơn độ phức tạp phần cứng.

Nhóm **INT Scheduler** phụ trách các lệnh số nguyên và một phần lệnh nhánh, địa chỉ, logic cơ bản. Nhóm **FP Scheduler** phụ trách các lệnh vector và dấu chấm động. Nhóm **MEM Scheduler** phụ trách các lệnh tải/lưu. Mỗi miền có đặc tính riêng. Lệnh số nguyên thường rất nhiều, tương đối ngắn và xuất hiện khắp nơi. Lệnh FP/vector thường có throughput cao nhưng cần datapath lớn hơn. Lệnh nhớ lại phải theo dõi phụ thuộc địa chỉ, phụ thuộc dữ liệu và độ trễ cache. Tách riêng các miền này là cách hợp lý để đạt hiệu năng cao mà không làm một khối trung tâm trở nên quá tải.

Trong số các scheduler, **128 MEM Scheduler** là cực kỳ nổi bật. Con số này cho thấy phần bộ nhớ không bị xem như một đường phụ, mà được đầu tư quy mô lớn. Điều này rất hợp lý, bởi trong các ứng dụng hiện đại, nghẽn bộ nhớ thường là một trong những nguyên nhân lớn nhất khiến IPC thực tế thấp hơn nhiều so với lý thuyết.

9. Miền thực thi số nguyên: xử lý logic, nhánh và số học nguyên

Bên dưới các INT Scheduler là các execution units số nguyên, được gắn nhãn như **ALU Branch**, **ALU**, **ALU MUL**, và **ALU MUL DIV**. Cách tổ chức này cho thấy miền integer vừa có **các đơn vị đa dụng**, vừa có **các đơn vị chuyên trách**.

Các khối **ALU** thông thường chịu trách nhiệm cho những phép toán cơ bản như cộng, trừ, dịch, so sánh, logic bit, tính địa chỉ đơn giản và nhiều dạng thao tác phổ biến khác. Những lệnh kiểu này có mặt với mật độ rất cao trong hầu hết chương trình. Nếu số ALU không đủ, hiệu năng tổng thể của core sẽ nhanh chóng bị giới hạn.

Các khối **ALU Branch** cho thấy một số execution units được tối ưu thêm cho đường xử lý nhánh. Điều này hợp lý vì nhánh không chỉ cần predictor tốt, mà còn cần được xử lý nhanh ở backend để xác nhận hoặc hiệu chỉnh kết quả dự đoán. Trong các pipeline sâu và rộng, branch latency có ảnh hưởng rất lớn đến hiệu năng, nên việc dành tài nguyên riêng cho loại lệnh này có thể giúp giảm độ trễ điều khiển.

Các khối **ALU MUL** và **ALU MUL DIV** thể hiện rằng miền integer cũng được trang bị cho các phép toán nặng hơn như nhân và chia. Nhân thường có throughput và latency khác với ALU thường; chia còn nặng hơn nữa và thường cần phần cứng riêng. Việc tách chúng ra giúp những lệnh phức tạp không làm tắc các cổng ALU phổ thông.

Tổng thể, phần integer của core này không chỉ rộng về số lượng mà còn được tổ chức để phù hợp với tính chất không đồng đều của workload thực tế. Điều này cho phép CPU vừa chạy tốt các đoạn mã điều khiển, vừa xử lý ổn định các phần tính toán số nguyên nặng hơn.

10. Miền vector và dấu chấm động: năng lực tính toán dữ liệu cao

Bên dưới các **52 FP Scheduler** là một dãy dài các khối mang nhãn **VALU**, **VMUL**, **FP ADD**, và phía dưới nữa là các khối **FP MUL** **FP DIV**. Đây là dấu hiệu rõ ràng cho thấy core này có một miền vector/dấu chấm động rất mạnh.

Trong các hệ thống ARM hiện đại, khối vector/FP thường phục vụ nhiều loại tác vụ quan trọng như xử lý đa phương tiện, mã hóa, nén, xử lý tín hiệu, thư viện toán học, machine learning truyền thống và các đoạn tối ưu SIMD trong phần mềm hệ thống. Một core có nhiều scheduler FP và nhiều cổng thực thi vector/FP sẽ có khả năng duy trì throughput cao với các phép cộng, nhân, tích lũy, và các phép biến đổi dữ liệu song song.

Điểm đáng chú ý là số lượng **FP Scheduler** lên tới 6 khối, mỗi khối **52 entry**, cho thấy không chỉ execution units nhiều mà **hàng đợi chờ và lập lịch cho lệnh FP/vector cũng rất sâu**. Điều này có lợi khi chương trình có nhiều lệnh vector đang chờ dữ liệu, hoặc khi backend cần giữ và sắp xếp một lượng lớn thao tác vector để tối ưu việc sử dụng cổng thực thi.

Sự hiện diện của **Vector PRF** riêng với quy mô lớn cũng cho thấy miền này là một phần trung tâm của thiết kế, chứ không phải chỉ là một phần bổ sung. Với các core nhỏ, phần vector thường bị hạn chế để tiết kiệm năng lượng. Ở đây thì ngược lại, toàn bộ sơ đồ cho thấy vector/FP được đầu tư ngang tầm với integer và memory.

11. Miền bộ nhớ: nơi quyết định khả năng nuôi dữ liệu cho backend

Nếu front-end là nơi nuôi lệnh cho backend, thì **memory subsystem** là nơi nuôi dữ liệu cho backend. Trong các CPU hiệu năng cao, rất nhiều lần sụt IPC không đến từ thiếu execution

units mà đến từ việc thiếu dữ liệu đúng lúc. Chính vì vậy, sơ đồ của P Core dành một phần rất lớn cho miền bộ nhớ.

Ngay dưới **128 MEM Scheduler**, sơ đồ có **2 AGU Store**, **2 AGU Unified**, và **4 AGU Load Data**. AGU là đơn vị tạo địa chỉ hiệu dụng cho lệnh truy cập bộ nhớ. Có thể hiểu đơn giản rằng trước khi một lệnh load hoặc store chạm tới cache, CPU phải tính được địa chỉ mà nó cần đọc hoặc ghi. Nếu không có đủ AGU, dù pipeline có nhiều lệnh nhớ đi nữa, CPU cũng không thể phát hành chúng đủ nhanh.

Việc có tới **4 AGU Load Data** cho thấy core thiên về tối ưu phần **load**, tức phần đọc dữ liệu. Đây là lựa chọn hợp lý vì trong nhiều chương trình, số lệnh đọc dữ liệu thường nhiều hơn số lệnh ghi. Bên cạnh đó, **2 AGU Store** chuyên trách cho đường ghi giúp việc lưu dữ liệu không phải tranh chấp hoàn toàn với load. **2 AGU Unified** bổ sung tính linh hoạt, có thể được sử dụng để phục vụ nhiều loại truy cập tùy thiết kế cụ thể.

Ngay dưới AGU là **96 Store Buffer** và **160 Load Buffer**. Đây là hai thành phần rất quan trọng nhưng thường bị người mới học kiến trúc bỏ qua. Store buffer giữ các lệnh ghi đã hoàn tất một phần nhưng chưa cần hoặc chưa thể ghi ngay xuống cache hay bộ nhớ sâu hơn. Điều này cho phép pipeline phía trước tiếp tục làm việc mà không phải dừng lại chỉ vì việc ghi chưa hoàn tất. Load buffer giữ các lệnh đọc đang chờ dữ liệu, đang chờ xác minh phụ thuộc, hoặc đang chờ phản hồi từ cache. Khi các buffer này lớn, CPU có thể duy trì nhiều truy cập bộ nhớ đồng thời hơn, từ đó tăng khả năng che giấu độ trễ.

Con số **160 Load Buffer** đặc biệt ấn tượng. Nó cho thấy core này được chuẩn bị để theo đuổi rất nhiều lệnh load cùng lúc, điều cực kỳ quan trọng khi chạy các chương trình có nhiều truy cập dữ liệu và độ trễ cache không phải lúc nào cũng thấp. Có thể nói, nếu predictor và ROB là thứ giúp core “nhìn xa” trong luồng lệnh, thì load/store buffers là thứ giúp core “chịu đựng tốt” hơn trước độ trễ của hệ thống nhớ.

12. L1 D-cache, TLB và các tầng cache tiếp theo

Ở phần gần execution nhất, sơ đồ cho thấy **128KB L1 D-Cache (8-way)** và **L1 DTLB 384 entry**. Đây là tầng cache dữ liệu gần nhất với các lệnh load/store. Dung lượng **128KB** cho L1D là lớn, nhất là với một cache có độ trễ phải được giữ thật thấp. Dung lượng lớn như vậy giúp tăng xác suất dữ liệu cần thiết đang có sẵn ngay tại L1, từ đó giảm chi phí truy cập.

Việc **L1 DTLB có 384 entry** cho thấy phần dịch địa chỉ dữ liệu cũng được đầu tư mạnh. Trong các hệ thống dùng bộ nhớ ảo, mỗi lần truy cập dữ liệu không chỉ là đi tìm dữ liệu trong cache mà còn là vấn đề dịch địa chỉ ảo sang địa chỉ vật lý. Nếu TLB nhỏ hoặc hit-rate thấp, truy cập bộ nhớ có thể bị chậm đáng kể. Với một core nhắm đến throughput cao, TLB lớn là điều rất cần thiết.

Sơ đồ còn ghi **pL2 Cache 1.5MB, pL2 TLB 640 entry, L2 TLB 4096 entry, và 16MB Shared L2 Cache (16-way)**. Dù cách đặt tên “pL2” và “Shared L2” có thể là quy ước riêng của nguồn sơ đồ, ý nghĩa tổng quát rất rõ: sau L1D, core còn có thêm những tầng đệm lớn hơn và sâu hơn để giảm số lần phải đi xuống bộ nhớ xa.

Một tầng cache riêng cỡ **1.5MB** ngay gần core có tác dụng như một vùng đệm trung gian giữa L1D rất nhanh nhưng nhỏ và tầng cache chia sẻ lớn hơn. Tầng **16MB shared cache** giúp giữ working set lớn hơn cho nhiều nhân hoặc cho toàn cụm xử lý. Với những chương trình có dữ liệu lớn, cấu trúc nhiều tầng như vậy rất quan trọng vì nó làm giảm mức độ “rơi thẳng xuống DRAM”, vốn có độ trễ rất cao.

Nhìn tổng thể, hệ thống bộ nhớ của core này không chỉ lớn ở dung lượng mà còn đồng bộ với front-end và backend rộng. Nói cách khác, kiến trúc không muốn rơi vào tình huống “decode được rất nhiều, issue được rất nhiều, nhưng cuối cùng vẫn phải đứng chờ memory”.

13. Dòng chảy hoạt động của toàn core

Để người đọc dễ hình dung, có thể mô tả core này như một chuỗi hoạt động liên hoàn.

Đầu tiên, **Branch Predictors** dự đoán địa chỉ lệnh tiếp theo. Nhờ predictor nhiều tầng, core có thể giữ cho luồng nạp lệnh liên tục ngay cả khi chương trình có nhiều nhánh. Sau đó, **Instruction Fetch Unit** lấy lệnh từ **L1 I-cache**, sử dụng **L1 ITLB** để hỗ trợ dịch địa chỉ, rồi đưa tối đa **64 bytes** mỗi chu kỳ xuống **Predecode Fetch Buffer**.

Từ đây, byte lệnh được đưa qua **12 Simple Decoders** hoặc lấy trực tiếp từ **Op-Cache** nếu đoạn mã đó đã được lưu ở dạng thao tác nội bộ. Kết quả đi vào **Decode Queue** với bề rộng tới **12 MOPs**. Tiếp theo, core thực hiện **register renaming**, gán thanh ghi vật lý từ **Integer PRF** hoặc **Vector PRF**, theo dõi vòng đời tài nguyên qua **PRRT**, và đưa lệnh vào **ROB 620 entry** để chờ thực thi và retirement theo đúng thứ tự kiến trúc.

Khi các toán hạng đã sẵn sàng và scheduler xác định có execution unit phù hợp, các vi lệnh được đưa qua **Issue 24 μops** tới các cụm scheduler integer, FP và memory. Lệnh số nguyên đi tới các **ALU**, **ALU Branch**, **ALU MUL**, **ALU MUL DIV**. Lệnh vector và FP đi tới các đơn vị **VALU**, **VMUL**, **FP ADD**, **FP MUL**, **FP DIV**. Lệnh bộ nhớ đi qua **AGU**, **load/store buffers**, rồi truy cập **L1 D-cache**, TLB và các tầng cache tiếp theo nếu cần.

Cuối cùng, khi các lệnh đã thực thi xong và mọi điều kiện phụ thuộc, ngoại lệ, thứ tự đều hợp lệ, chúng được **retire** với tốc độ tối đa **16 μops mỗi chu kỳ**. Chính tại bước này, trạng thái kiến trúc của CPU được cập nhật chính thức.

Nếu nhìn theo dòng chảy đó, có thể thấy lý do vì sao sơ đồ này được xem là một performance core. Mỗi giai đoạn đều được thiết kế để hạn chế việc trở thành nút cổ chai cho giai đoạn kế tiếp.

14. Vì sao sơ đồ này cho thấy đây là một core hiệu năng cao

Không cần biết chính xác tên thương mại của core, chỉ dựa vào các con số trong sơ đồ cũng có thể nhận ra đây là một nhân có đẳng cấp cao về hiệu năng.

Thứ nhất, **predictor nhiều tầng và rất lớn** cho thấy nhà thiết kế sẵn sàng chi nhiều tài nguyên chỉ để bảo đảm front-end không bị đứt quãng bởi nhánh. Đây là lựa chọn chỉ thực sự hợp lý ở các core rộng, nơi chi phí của branch misprediction rất lớn.

Thứ hai, **decode 12-wide**, có **op-cache**, fetch **64 bytes**, và **I-cache lớn** cho thấy front-end này không chỉ có độ chính xác mà còn có **throughput cấp lệnh rất cao**. Điều này đặc biệt quan trọng đối với hiệu năng đơn luồng.

Thứ ba, **ROB 620 entry**, **PRF lớn**, **PRRT lớn** là những dấu hiệu không thể nhầm của một kiến trúc out-of-order sâu. Một core như vậy có thể giữ rất nhiều lệnh trong hệ thống, tìm kiếm song song lệnh tốt hơn và che giấu độ trễ hiệu quả hơn.

Thứ tư, **Issue 24 μops** và **Retire 16 μops** là những bề rộng rất lớn. Chúng cho thấy backend không hề bị giới hạn ở mức trung bình mà được thiết kế để hấp thụ và hoàn thành một lượng lớn công việc trong mỗi chu kỳ.

Thứ năm, **số lượng scheduler và execution units nhiều** ở cả integer lẫn FP/vector cho thấy core không chỉ mạnh với mã thông thường mà còn mạnh cả với các workload tính toán dữ liệu.

Thứ sáu, **128 MEM scheduler, 8 AGU, 160 load buffer, 96 store buffer**, cùng hệ thống cache/TLB lớn, là bằng chứng rõ ràng rằng nhà thiết kế hiểu rằng hiệu năng CPU hiện đại không thể chỉ đến từ ALU. Để thực sự nhanh, core phải có khả năng “đuổi theo” rất nhiều truy cập bộ nhớ mà không bị nghẹt quá sớm.

15. Core này phù hợp với loại công việc nào

Từ cách tổ chức phần cứng, có thể hình dung core này đặc biệt phù hợp với những loại tải công việc cần **hiệu năng đơn luồng cao**, nhiều nhánh, nhiều lệnh hỗn hợp, và mức song song nội tại lớn. Những tác vụ như ứng dụng desktop phức tạp, trình duyệt, môi trường runtime, các engine ứng dụng, logic game, mã biên dịch, thư viện hệ thống, hay các đoạn mã native hiệu năng cao đều có thể hưởng lợi từ front-end mạnh, predictor tốt và cửa sổ OOO sâu.

Đồng thời, nhờ cụm vector/FP mạnh, core này cũng phù hợp với các bài toán xử lý dữ liệu, multimedia, tín hiệu số, mã hóa, nén và các phần mềm có dùng tăng tốc SIMD. Còn nhờ memory subsystem lớn, nó cũng có khả năng đối phó tốt hơn với những workload có nhiều truy cập dữ liệu, working set khá lớn hoặc độ trễ bộ nhớ khó đoán.

Tất nhiên, không thể chỉ từ sơ đồ mà khẳng định hiệu năng thực tế trên từng ứng dụng cụ thể, bởi điều đó còn phụ thuộc tần số hoạt động, phần mềm, compiler, hệ điều hành, độ trễ cache thực, băng thông DRAM và rất nhiều yếu tố khác. Nhưng xét về mặt vi kiến trúc, đây rõ ràng là một core được chuẩn bị để **không dễ bị hụt hơi** trước nhiều loại tải nặng.

16. Kết luận

P Core trong sơ đồ là một ví dụ điển hình của một **nhân CPU ARM hiệu năng cao** được thiết kế với tư duy rất hiện đại và rất quyết liệt về mặt vi kiến trúc. Toàn bộ front-end được mở rộng với predictor nhiều tầng, l-cache lớn, băng thông fetch rộng, dàn decode 12-wide và op-cache. Phần trung tâm của thực thi ngoài thứ tự được củng cố bởi PRF lớn, PRRT lớn và ROB sâu tới 620 entry. Backend lại được chia thành nhiều scheduler chuyên biệt, với

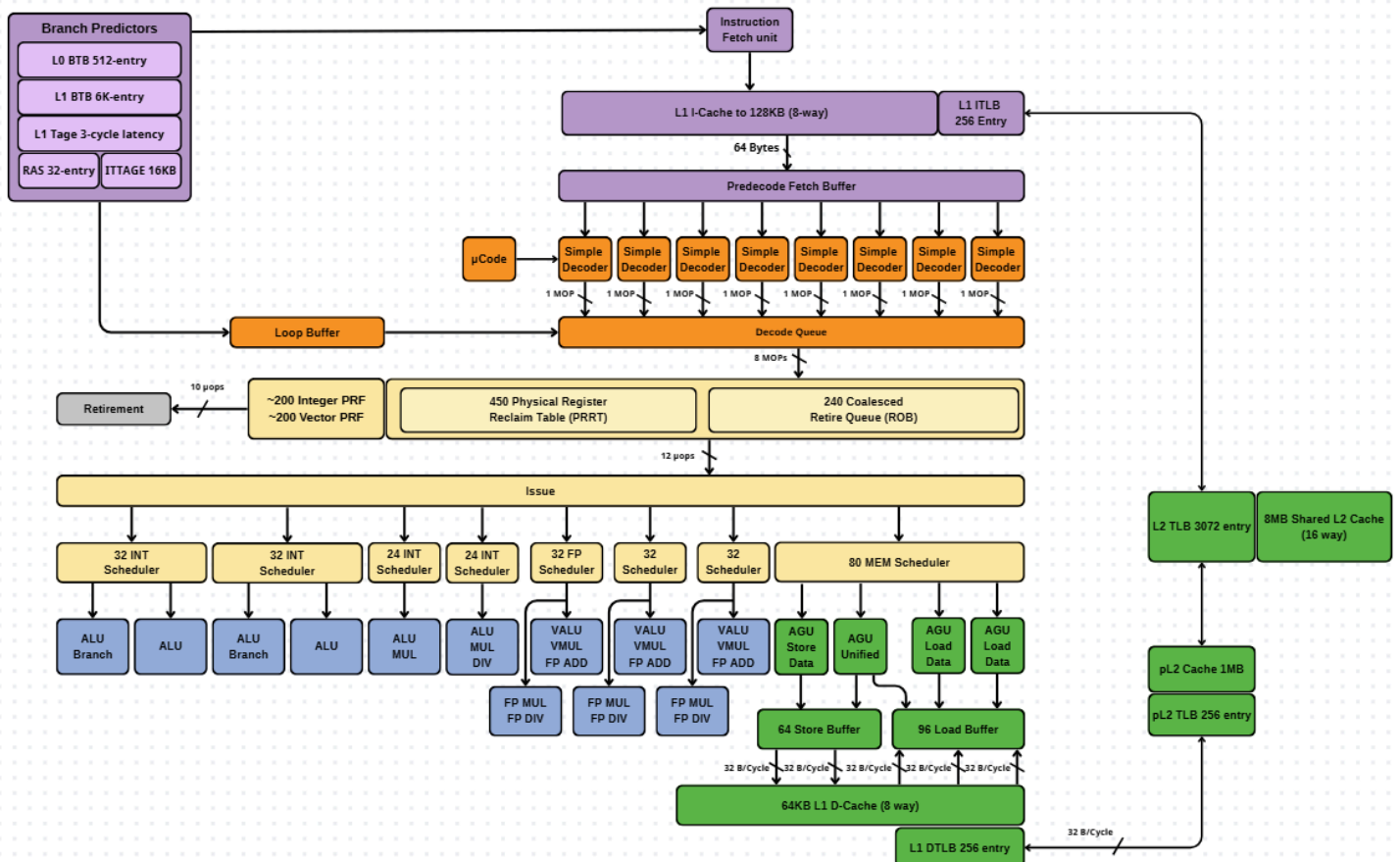
cum integer, vector/FP và memory đều rất mạnh. Cuối cùng, hệ thống bộ nhớ với AGU nhiều, load/store buffers lớn, TLB lớn và cache nhiều tầng bảo đảm rằng khả năng xử lý của core không bị bóp nghẹt quá sớm bởi độ trễ dữ liệu.

Nếu cần mô tả ngắn gọn cho người khác, có thể nói rằng đây là một **core cực rộng, cực sâu, và được cân bằng rất tốt giữa cấp lệnh, thực thi và cấp dữ liệu**. Nó không chỉ nhanh vì có nhiều ALU, mà nhanh vì mọi tầng trong pipeline đều được đầu tư để hỗ trợ lẫn nhau. Chính sự đồng bộ đó là dấu hiệu quan trọng nhất cho thấy đây là một **performance core đúng nghĩa**.

Nếu bạn muốn, ở bước tiếp theo mình có thể chuyển ngay bài này thành một trong ba dạng sau: một **bản báo cáo kỹ thuật chuẩn đề nộp**, một **bản thuyết minh dễ đọc hơn như tài liệu giới thiệu kiến trúc**, hoặc một **bản có thêm sơ đồ giải thích từng khối theo phong cách slide trình bày**.

C Core

C Core



Báo cáo kỹ thuật về kiến trúc **C Core** trong sơ đồ đã cung cấp

1. Mở đầu

Nếu nhìn kiến trúc này trong tương quan với một hệ xử lý đã có **P Core** đảm nhiệm vai trò dẫn dắt về hiệu năng đơn nhân, thì bản chất của **C Core** cần được hiểu theo một cách khác. Đây không phải là kiểu nhân được xây dựng để cạnh tranh trực diện ở đỉnh hiệu năng của một luồng đơn lẻ, cũng không phải là phiên bản “thu nhỏ đơn giản” của P Core. Ngược lại, sơ đồ cho thấy C Core là một thiết kế có chủ đích rất rõ: tối ưu cho **khả năng mở rộng đa luồng, gia tăng sức mạnh đa nhân**, và nâng cao **thông lượng xử lý tổng thể** của toàn bộ bộ xử lý khi nhiều lõi cùng tham gia làm việc.

Điểm quan trọng nhất của C Core nằm ở triết lý phân vai. Trong một kiến trúc lai, **P Core** gánh phần công việc đòi hỏi độ trễ thấp, phản hồi nhanh, IPC đơn luồng cao và sức mạnh tức thời cho các luồng quan trọng nhất. Còn **C Core** được tổ chức để xử lý khối lượng công việc lớn hơn theo hướng song song hóa: nhiều luồng nền, nhiều tác vụ đồng thời, nhiều tiến trình độc lập, các pipeline dữ liệu chạy dài, và các workload có thể chia thành nhiều phần tương đối tách biệt. Vì vậy, khi phân tích sơ đồ này, điều cần nhấn mạnh không phải là “một C Core mạnh cỡ nào nếu đứng riêng lẻ”, mà là “cụm nhiều C Core có thể đóng góp ra sao cho sức mạnh tổng thể của hệ thống”.

Sơ đồ thể hiện một vi kiến trúc vẫn rất nghiêm túc về mặt tổ chức: front-end rộng, branch prediction nhiều tầng, decode song song, cơ chế đổi tên thanh ghi, cửa sổ out-of-order đủ sâu, các scheduler phân cụm, nhiều nhóm execution unit, cùng một hệ thống bộ nhớ và cache được đầu tư bài bản. Tất cả cho thấy đây là một lõi được thiết kế để vừa đủ mạnh ở cấp độ từng nhân, vừa đủ hợp lý về mặt tài nguyên để có thể **nhân lên nhiều bản sao trong cùng một thiết kế SoC hay CPU**, qua đó tạo ra giá trị thật sự ở cấp độ đa nhân.

2. Định vị kiến trúc: từ hiệu năng cục bộ sang sức mạnh tổng hợp

Điểm khác biệt cốt lõi giữa C Core và P Core không nằm ở việc cái nào “hiện đại hơn” hay “nhiều tính năng hơn”, mà nằm ở **mục tiêu tối ưu**. P Core được xây dựng để theo đuổi hiệu năng đỉnh cho những luồng quan trọng nhất. Trong khi đó, C Core được xây dựng để đạt **mật độ hiệu năng cao hơn trên toàn chip**, nghĩa là với một ngân sách diện tích và điện năng hợp lý, hệ thống có thể tích hợp nhiều nhân hơn, từ đó xử lý được nhiều luồng hơn và nâng tổng hiệu năng đa nhân lên rõ rệt.

Chính vì vậy, khi nhìn vào C Core, ta không nên dùng thước đo đơn thuần là “luồng đơn chạy nhanh đến mức nào” để đánh giá toàn bộ giá trị của nó. Giá trị thực của kiến trúc này nằm ở khả năng trở thành một “đơn vị xử lý hàng loạt” có năng lực tốt, hoạt động ổn định, và khi được triển khai theo cụm hoặc theo số lượng lớn, nó sẽ cộng lực thành một nền hiệu năng rất đáng kể. Với cách tiếp cận này, C Core không thay thế P Core mà **bổ sung cho P Core**, tạo nên thế cân bằng giữa đỉnh hiệu năng đơn nhân và tổng lực đa nhân.

Sơ đồ cho thấy C Core vẫn là một kiến trúc có tham vọng kỹ thuật rõ rệt. Nó không phải nhân tối giản kiểu chỉ vừa đủ chạy tác vụ nền. Việc có predictor nhiều lớp, decode rộng tới 8 đơn vị, ROB 240 mục, issue 12 μops, retirement 10 μops, scheduler bộ nhớ lớn, cùng nhiều execution unit integer, vector và FP cho thấy C Core vẫn được xây như một “core thực thụ”, chỉ khác ở chỗ trọng tâm không đặt vào việc vượt P Core ở đơn luồng, mà đặt vào **khả năng duy trì hiệu suất tốt trên nhiều luồng và nhiều lõi đồng thời**.

3. Cái nhìn tổng thể về dòng chảy kiến trúc

Xét theo luồng xử lý, C Core có cấu trúc rất rõ ràng. Pipeline bắt đầu từ hệ **branch prediction**, chuyển sang **instruction fetch**, sau đó đi qua **L1 instruction cache**, **instruction TLB**, **predecode fetch buffer**, cụm **8 simple decoder** và đường **μCode** dành cho các lệnh phức tạp. Từ đây, các vi thao tác được đưa vào **decode queue**, đi qua các khối quản lý tài nguyên như **physical register file**, **physical register reclaim table**, **retire queue/ROB**, rồi tiến vào giai đoạn **issue**.

Sau issue, vi lệnh được phân phối tới nhiều **scheduler** riêng cho integer, floating-point/vector và memory. Từ scheduler, chúng được đưa tới các **execution unit** tương ứng như ALU, branch unit, đơn vị nhân/chia nguyên, các khối VALU/VMUL/FP, cùng cụm AGU dành cho thao tác địa chỉ bộ nhớ. Phần cuối của pipeline là hệ thống **load/store**

buffer, **L1 data cache**, các tầng **TLB** và **L2 cache**, trước khi lệnh đủ điều kiện để **retire** theo thứ tự kiến trúc.

Điểm đáng chú ý là gần như mọi giai đoạn trong pipeline này đều được tổ chức với băng thông tương đối lớn. Điều đó phản ánh một triết lý nhất quán: mỗi C Core phải đủ mạnh để không trở thành một nhân “mỏng”, bởi nếu từng nhân quá yếu thì lợi ích của việc tăng số lõi sẽ bị giảm đáng kể. Nói cách khác, đây là một kiến trúc xây cho **scale-out**, nhưng không hy sinh đến mức làm mất đi năng lực xử lý thực chất của từng core.

4. Front-end và vai trò của branch prediction trong một lõi hướng đa nhân

Khối đầu tiên gây ấn tượng trong sơ đồ là **Branch Predictors**. Tại đây có **L0 BTB 512-entry**, **L1 BTB 6K-entry**, **L1 Stage 3-cycle latency**, **RAS 32-entry** và **ITTAGE 16KB**. Chỉ riêng tổ hợp này đã cho thấy front-end của C Core không hề sơ sài. Nó được trang bị một hệ dự đoán nhánh nhiều tầng, đủ sâu và đủ rộng để phục vụ những dòng lệnh phức tạp, nhiều nhánh, nhiều vòng lặp và nhiều call/return.

Trong các CPU hiện đại, branch prediction có vai trò sống còn vì pipeline càng rộng thì cái giá của dự đoán sai càng lớn. Với C Core, điều này lại càng quan trọng trong bối cảnh đa nhân. Lý do là khi một hệ thống có nhiều lõi cùng hoạt động, hiệu năng tổng thể không chỉ phụ thuộc vào việc có bao nhiêu lõi, mà còn phụ thuộc vào việc mỗi lõi có giữ được trạng thái “được nuôi lệnh đều đặn” hay không. Nếu predictor yếu, pipeline phía sau sẽ thường xuyên bị đói lệnh hoặc bị flush vì nhánh sai, khiến hiệu quả thực tế của cụm đa nhân giảm xuống đáng kể.

L0 BTB phục vụ những trường hợp dự đoán rất nhanh ở lớp gần nhất, trong khi **L1 BTB 6K-entry** mở rộng độ bao phủ cho nhiều đích nhảy hơn. **RAS 32-entry** là cấu trúc rất quan trọng đối với chuỗi call/return, đặc biệt trong mã ứng dụng hiện đại vốn có nhiều lớp hàm, nhiều thư viện và mức độ trừu tượng cao. **ITTAGE 16KB** cho thấy core có một predictor tương đối mạnh, thiên về việc học các mẫu nhánh phức tạp thay vì chỉ dựa vào những cơ chế đơn giản.

Điểm đáng nói là ở C Core, predictor mạnh không chỉ để “đẩy IPC đẹp” cho một luồng đơn, mà còn để đảm bảo **tính ổn định thông lượng** trên từng nhân. Một nhân có thể không

được ưu tiên tối đa như P Core, nhưng vẫn phải duy trì front-end đủ sắc bén để khi hàng loạt C Core cùng chạy, cả cụm không bị thất thoát hiệu năng vì sai nhánh quá nhiều.

5. Instruction fetch, I-Cache và tầng đệm trước decode

Sau khối predictor là **Instruction Fetch unit**, nối xuống **L1 I-Cache to 128KB (8-way)**. Mức dung lượng **128KB** cho cache lệnh cấp L1 là tương đối rộng, cho phép giữ được lượng mã “nóng” lớn hơn và giảm áp lực phải đi xuống tầng cache sâu hơn. Thiết kế **8-way** cũng giúp hạn chế xung đột ánh xạ, điều rất hữu ích khi nhiều đoạn mã được truy cập luân phiên hoặc khi ứng dụng có vùng code lớn.

Sơ đồ còn ghi **64 Bytes** trên đường nạp lệnh xuống dưới, cho thấy mỗi nhịp fetch có thể đưa về một lượng byte lệnh tương đối lớn. Trong các pipeline hiện đại, băng thông fetch như vậy rất cần thiết nếu phía sau là một khối decode rộng. Nếu fetch không đủ khỏe thì số lượng decoder nhiều đến đâu cũng khó được khai thác hết.

Ở đây, dữ liệu lệnh được đưa qua **Predecode Fetch Buffer** trước khi đi tới decoder. Tầng đệm này có ý nghĩa rất thực tiễn. Lệnh máy trong nhiều ISA thường có độ dài không đồng đều hoặc có cách mã hóa cần được “chuẩn hóa” trước khi decode thực sự diễn ra.

Predecode buffer giúp tách rời phần nạp byte lệnh khỏi phần giải mã cấu trúc lệnh, từ đó làm front-end ổn định hơn và giảm nguy cơ nghẽn giữa fetch với decode.

Bên cạnh đó còn có **L1 ITLB 256 entry**, hỗ trợ dịch địa chỉ lệnh ở mức gần nhất. Sự hiện diện của **L2 TLB 3072 entry** ở phía sâu hơn cho thấy front-end không chỉ được đầu tư về cache, mà còn được bảo vệ trước áp lực của việc dịch địa chỉ trong những không gian mã lớn. Đối với một kiến trúc hướng đa nhân, điều này càng có giá trị, vì khi số luồng và số tiến trình tăng lên, xác suất làm việc trên nhiều vùng bộ nhớ mã khác nhau cũng tăng theo.

6. Giải mã lệnh: rộng, chắc, và phục vụ thông lượng bền vững

Một trong những điểm nổi bật nhất của sơ đồ là cụm giải mã gồm **8 Simple Decoder**, mỗi decoder gắn nhãn **1 MOP**, cùng với một đường **μCode** độc lập. Sau decode, toàn bộ vi thao tác được đẩy vào **Decode Queue**, với chú thích **8 MOPs**.

Về mặt kỹ thuật, đây là một front-end decode khá rộng. Nó cho phép C Core tiếp nhận được nhiều lệnh đồng thời ở những trường hợp lệnh thuộc dạng đơn giản, dễ tách và dễ chuyển thành vi thao tác. Việc bổ sung đường **µCode** cho thấy những lệnh phức tạp không cần kéo chậm toàn bộ đường decode chính; thay vào đó, chúng có thể được xử lý bằng con đường riêng phù hợp hơn.

Tuy nhiên, ý nghĩa sâu hơn của cụm decode này trong C Core không phải là chứng minh rằng core muốn tranh vị trí “vua đơn nhân”, mà là bảo đảm rằng **mỗi core trong cụm đa nhân đều có front-end đủ mạnh để không bị tụt thông lượng**. Nếu một lõi hướng đa nhân có decode quá hẹp, nó sẽ nhanh chóng bộc lộ điểm yếu khi chạy các thread thực tế có trộn nhiều kiểu lệnh khác nhau. Khi đó, việc tăng số lõi chưa chắc đem lại tỷ lệ tăng hiệu năng tương xứng.

Sự hiện diện của **Loop Buffer** cũng rất đáng chú ý. Đây là cấu trúc có giá trị lớn với các vòng lặp ngắn và lặp nhiều lần, vốn rất phổ biến trong thread worker, xử lý dữ liệu, mã runtime, mã thư viện, các tác vụ nền hoặc những vòng xử lý khối trong multimedia. Loop buffer giúp tránh việc phải liên tục quay lại toàn bộ đường fetch và decode cho cùng một cụm lệnh lặp lại, từ đó giảm áp lực lên front-end và cải thiện hiệu quả năng lượng cũng như thông lượng.

Nhìn từ góc độ hệ thống, đây là biểu hiện của một thiết kế có suy nghĩ dài hạn: mỗi C Core phải chạy tốt không chỉ ở benchmark ngắn, mà còn ở những workload kéo dài, có tính lặp và phân bố rộng trên nhiều nhân.

7. Rename, PRF, PRRT và ROB: nền móng của thực thi ngoài thứ tự

Sau decode queue, sơ đồ đi vào vùng trung tâm của pipeline với các cấu trúc **~200 Integer PRF**, **~200 Vector PRF**, **450 Physical Register Reclaim Table (PRRT)** và **240 Coalesced Retire Queue (ROB)**. Đây là những thành phần cốt lõi của một CPU out-of-order hiện đại.

Việc có **integer PRF** và **vector PRF** tách biệt cho thấy core quản lý hai miền dữ liệu theo cách độc lập và chuyên biệt. Điều này rất hợp lý với một kiến trúc có cả integer path mạnh lẫn vector/FP path khá phong phú. Khi dữ liệu scalar và vector được theo dõi ở những

không gian vật lý riêng, kiến trúc có thể tối ưu tốt hơn việc cấp phát, tái sử dụng và thu hồi tài nguyên thanh ghi.

PRRT 450 là một con số đáng chú ý vì nó gợi ý khả năng quản lý số lượng lớn ánh xạ và vòng đời của thanh ghi vật lý. Còn **ROB 240** cho thấy cửa sổ out-of-order của C Core là tương đối sâu. Một cửa sổ như vậy cho phép rất nhiều lệnh cùng tồn tại trong pipeline, chờ dữ liệu, chờ giải quyết phụ thuộc hoặc chờ điều kiện commit. Với một lõi hướng đa nhân, điều này cực kỳ quan trọng, bởi các thread thực tế hiếm khi hoàn hảo và không phải lúc nào cũng là những dòng lệnh đơn giản, sạch phụ thuộc.

Một C Core muốn có giá trị trong hệ thống nhiều lõi phải đủ mạnh để tự che giấu phần nào các độ trễ cục bộ, đặc biệt là độ trễ bộ nhớ, độ trễ nhân/chia, hoặc các chuỗi phụ thuộc dữ liệu ngắn. Nếu không có ROB đủ sâu và cơ chế rename đủ rộng, lõi sẽ dễ bị “đờ nhíp” khi gặp mã không đẹp. Rõ ràng sơ đồ này cho thấy nhà thiết kế không chấp nhận kiểu nhân “số lượng nhiều nhưng quá nông”, mà muốn mỗi C Core vẫn là một đơn vị xử lý có chiều sâu và có sức chịu tải tốt.

8. Issue và retirement: băng thông lớn để giữ nhịp cho toàn pipeline

Sơ đồ ghi rõ băng thông **Issue 12 μ ops** và **Retirement 10 μ ops**. Đây là những con số rất đáng kể. Về mặt tổng quan, một kiến trúc có thể decode rộng, có ROB sâu, nhưng nếu issue hoặc retire hẹp thì hiệu quả thực tế sẽ bị bóp nghẹt ở một trong hai đầu vào và đầu ra của back-end. C Core tránh điều này bằng cách duy trì độ rộng tương xứng ở cả hai hướng.

Issue 12 μ ops cho phép core mỗi chu kỳ có thể phát đi một lượng vi thao tác lớn tới các scheduler khác nhau, miễn là dữ liệu sẵn sàng và không có xung đột tài nguyên. Đây là điều rất cần thiết trong một kiến trúc có nhiều cụm thực thi song song. Trong khi đó, **Retirement 10 μ ops** cho phép lệnh sau khi hoàn thành có thể được nghỉ hưu với tốc độ cao, giảm nguy cơ ùn tắc ở đầu ra kiến trúc.

Ở đây cần nhấn mạnh rằng các con số này không nên chỉ đọc theo kiểu “đơn nhân mạnh cỡ nào”, mà nên hiểu như một cam kết rằng **mỗi C Core đều có năng lực xử lý liên tục và bền**. Khi nhiều C Core cùng hoạt động, hiệu suất tổng hợp của hệ thống phụ thuộc rất lớn vào việc từng core có giữ được nhịp hay không. Một core chỉ mạnh ở vài giai đoạn mà yếu

ở các điểm nút quan trọng sẽ khó nhân rộng hiệu quả. C Core trong sơ đồ này cho thấy sự cân bằng tốt giữa decode, issue, execute và retire, điều rất phù hợp với mục tiêu mở rộng theo số lượng lõi.

9. Scheduler phân cụm: tổ chức hợp lý cho một lõi cần nhân rộng

Khối issue đổ vào nhiều scheduler riêng biệt, bao gồm **32 INT Scheduler**, **32 INT Scheduler**, **24 INT Scheduler**, **24 INT Scheduler**, **32 FP Scheduler**, hai cụm **32 Scheduler** khác, cùng **80 MEM Scheduler**. Cách phân chia này cho thấy C Core được tổ chức theo triết lý **phân vùng chức năng**, thay vì dồn tất cả vào một scheduler khổng lồ duy nhất.

Đây là một hướng đi rất hợp lý với một kiến trúc nhắm tới nhân rộng nhiều lõi. Một scheduler toàn cục quá lớn thường rất đắt đỏ về điện năng, độ phức tạp thiết kế và chi phí timing. Việc chia thành các cụm integer, vector/FP và memory riêng giúp giảm độ phức tạp của logic chọn lệnh, đồng thời cho phép từng nhánh thực thi phát triển theo đúng tính chất của loại tác vụ mà nó phục vụ.

Phía integer có hai cụm 32-entry và hai cụm 24-entry, cho thấy khối lượng công việc số nguyên vẫn là trọng tâm lớn của C Core. Điều này hoàn toàn hợp lý vì phần lớn các luồng ứng dụng thông thường, kể cả trong môi trường desktop lẫn server nhẹ, đều có mật độ integer, branch và address arithmetic cao. Bên cạnh đó, cụm **80 MEM Scheduler** là một chỉ dấu rất mạnh rằng kiến trúc này đặc biệt quan tâm đến thao tác bộ nhớ. Với các workload đa luồng, hiệu năng thường bị ràng buộc rất mạnh bởi load/store và khả năng xử lý phụ thuộc bộ nhớ. Một scheduler bộ nhớ lớn giúp core theo dõi và điều phối nhiều thao tác bộ nhớ đồng thời, từ đó giữ throughput ổn định hơn.

Nói một cách ngắn gọn, tổ chức scheduler trong C Core cho thấy đây là một kiến trúc **thiết thực**. Nó không chạy theo một khối khổng lồ duy nhất để phô trương độ rộng, mà chọn cách phân cụm để đạt hiệu quả tốt hơn khi được triển khai ở quy mô nhiều core.

10. Khối thực thi số nguyên: nền lực chính cho phần lớn tác vụ phổ thông

Ở phía integer execution, sơ đồ thể hiện các đơn vị **ALU Branch**, **ALU**, **ALU Branch**, **ALU**, **ALU MUL**, và **ALU MUL DIV**. Cách bố trí này cho thấy C Core có một hệ thực thi số nguyên khá dày và được chuyên môn hóa tương đối tốt.

Hai đơn vị **ALU Branch** cho phép xử lý nhánh và các phép so sánh điều khiển luồng với băng thông tốt hơn, giảm khả năng nghẽn ở những đoạn mã có nhiều điều kiện. Hai đơn vị **ALU** thông thường phụ trách các phép tính số nguyên cơ bản, logic, cộng trừ, dịch bit, cập nhật biến đếm và nhiều thao tác nền tảng khác. Các đơn vị **ALU MUL** và **ALU MUL DIV** tách riêng phép nhân, chia và các thao tác có độ trễ cao hơn khỏi đường ALU cơ bản, nhờ đó các lệnh integer phổ thông không phải tranh tài nguyên với các phép toán nặng hơn.

Đối với một lõi hướng tới đa luồng và đa nhân, cụm integer như vậy là cực kỳ quan trọng. Phần lớn luồng chạy trong hệ thống không phải đều là bài toán dấu chấm động nặng. Rất nhiều công việc thực tế xoay quanh điều khiển luồng, thao tác chỉ số, xử lý trạng thái, logic nghiệp vụ, runtime, mã hệ thống, mã backend hoặc các bước điều phối trong pipeline. Nếu khối integer yếu, lõi sẽ nhanh chóng bị giới hạn. C Core rõ ràng tránh nhược điểm đó bằng cách đầu tư cho integer path một cách nghiêm túc.

Chính khối integer này giúp C Core trở thành một “đơn vị lao động” hiệu quả của hệ thống. Nó không nhất thiết phải thắng P Core trong mọi bài test đơn luồng, nhưng nó đủ khỏe để khi có nhiều C Core cùng hoạt động, khối lượng công việc hoàn thành trên toàn chip tăng lên rất rõ rệt.

11. Vector và floating-point: không chỉ là nhân nền, mà còn là nhân làm việc nặng

Phần vector và dấu chấm động của sơ đồ gồm ba cụm mang nhãn **VALU / VMUL / FP ADD**, bên dưới là hai cụm **FP MUL / FP DIV**. Điều này cho thấy C Core không chỉ được tối ưu cho các tác vụ integer và điều khiển, mà còn có năng lực khá tốt đối với các workload được vector hóa hoặc cần xử lý số thực.

Sự kết hợp giữa các cụm VALU/VMUL/FP ADD với các đơn vị FP MUL/FP DIV cho phép core phục vụ nhiều dạng phép toán khác nhau: từ cộng số thực, nhân vector, tới các phép toán FP có độ trễ cao hơn. Khi đi cùng với **~200 Vector PRF**, có thể thấy miền vector/FP

không phải phần phụ, mà là một mảng được đầu tư đủ để tạo khác biệt trong các tác vụ phù hợp.

Ý nghĩa của điều này trong bối cảnh đa nhân rất lớn. Nhiều ứng dụng hiện đại không chỉ chia được theo thread, mà còn tận dụng SIMD bên trong từng thread. Khi đó, cụm C Core có thể đồng thời hưởng lợi từ hai tầng song song hóa: **song song theo lõi** và **song song theo dữ liệu**. Những tác vụ như xử lý hình ảnh, âm thanh, encode/decode, filter, nén dữ liệu, các thư viện số học, thậm chí một số pipeline inference nhẹ, đều có thể tận dụng được điều này.

Điểm đáng quý của C Core là nó không bị giới hạn ở vai trò “core chạy nền đơn giản”. Sơ đồ cho thấy đây là nhân có thể gánh khối lượng công việc nghiêm túc, chỉ khác ở chỗ nó phát huy giá trị lớn nhất khi được đặt vào thế **hiệu năng phối hợp**, thay vì bắt một nhân riêng lẻ phải giành ngôi hiệu năng đơn luồng số một.

12. Hệ thống bộ nhớ: chìa khóa thật sự của sức mạnh đa nhân

Nếu phải chọn phần thể hiện rõ nhất triết lý đa nhân của C Core, thì đó chính là hệ thống bộ nhớ. Ở phía execution, sơ đồ cho thấy có **AGU Store Data**, **AGU Unified**, **AGU Load Data**, **AGU Load Data**. Cấu trúc này cho thấy core có nhiều cổng địa chỉ bộ nhớ, trong đó thiên khá rõ về phía load, một lựa chọn hợp lý vì nhiều ứng dụng đọc dữ liệu nhiều hơn ghi dữ liệu.

Sau AGU là **64 Store Buffer** và **96 Load Buffer**. Đây là những cấu trúc đặc biệt quan trọng trong CPU out-of-order. Load buffer lớn cho phép core theo dõi nhiều thao tác đọc đang chờ, còn store buffer lớn giúp trì hoãn và quản lý ghi bộ nhớ một cách linh hoạt hơn. Với các workload đa luồng, đặc biệt là những workload dữ liệu lớn hoặc nhiều access pattern phức tạp, các buffer này đóng vai trò không nhỏ trong việc giữ nhịp pipeline.

Lớp nhớ gần nhất là **64KB L1 D-Cache (8 way)**, kèm theo **L1 DTLB 256 entry**. Dung lượng này đủ khỏe để phục vụ nhiều working set cục bộ trước khi phải đi xa hơn. Phía dưới là **pL2 Cache 1MB**, **pL2 TLB 256 entry**, **L2 TLB 3072 entry**, và đặc biệt là **8MB Shared L2 Cache (16 way)**. Cấu trúc nhiều tầng như vậy rất phù hợp với một cụm lõi cần chia sẻ tài nguyên và tái sử dụng dữ liệu giữa các thread liên quan.

Sự hiện diện của **Shared L2 Cache 8MB** là chi tiết mang ý nghĩa kiến trúc rất lớn. Một tầng cache chia sẻ như vậy thường giúp cải thiện đáng kể hiệu quả của nhóm lõi trong các kịch bản nhiều thread có dữ liệu gần nhau hoặc phụ thuộc nhau. Thay vì mỗi lõi chỉ sống khép kín với cache riêng, cụm C Core có thể tận dụng một không gian gần hơn để trao đổi và giữ dữ liệu dùng chung. Đây là điểm hết sức phù hợp với định hướng đa nhân: không chỉ tăng số lõi, mà còn làm cho các lõi đó **hợp tác hiệu quả hơn**.

Nói cách khác, nếu P Core đại diện cho đỉnh cục bộ, thì hệ thống nhớ của C Core đại diện cho **năng lực vận hành bền vững ở quy mô lớn**. Nhiều lõi muốn phát huy tác dụng thì phải được nuôi dữ liệu tốt. Sơ đồ cho thấy C Core được xây với nhận thức rất rõ về điều đó.

13. Dòng chảy hoạt động của core dưới góc nhìn hệ thống

Khi nhìn lại toàn bộ pipeline, có thể mô tả hoạt động của C Core như sau. Đầu tiên, hệ predictor nhiều tầng dự đoán hướng đi của luồng lệnh và cung cấp thông tin cho instruction fetch. Lệnh được lấy từ L1 I-cache, đi qua ITLB và predecode fetch buffer trước khi được phân phối tới tám simple decoder hoặc đường μ Code. Các vi thao tác tạo ra từ đó được đưa vào decode queue.

Tiếp theo, vi lệnh được đổi tên thành ghi, cấp phát tài nguyên vật lý, ghi nhận trong ROB và chuẩn bị cho giai đoạn issue. Từ đây, tối đa 12 μ ops mỗi chu kỳ có thể được đưa tới các scheduler integer, FP/vector và memory. Khi toán hạng đã sẵn sàng, các lệnh sẽ được thực thi trên các đơn vị ALU, branch, MUL/DIV, VALU, FP và AGU tương ứng. Những thao tác bộ nhớ đi qua load/store buffer rồi truy cập vào hệ thống cache và TLB nhiều tầng. Sau khi hoàn thành và không còn vi phạm thứ tự kiến trúc, lệnh được nghỉ hưu với băng thông tối đa 10 μ ops mỗi chu kỳ.

Nếu diễn giải đơn thuần ở cấp độ lõi, đây là một pipeline hiện đại, rộng và có chiều sâu. Nhưng nếu diễn giải đúng tinh thần của C Core, thì ý nghĩa lớn hơn là: **mỗi core được tạo ra để có thể duy trì pipeline ổn định, chịu được workload thực tế, và khi nhiều core như vậy cùng hiện diện, toàn bộ hệ thống sẽ có một nền thông lượng rất mạnh**.

14. Vai trò của C Core trong một kiến trúc lai có P Core

Trong một hệ xử lý lai, sự khác biệt giữa P Core và C Core nên được xem là sự khác biệt về **chức năng chiến lược**, không phải chỉ là sự khác biệt về “mạnh” hay “yếu”. **P Core** là nơi xử lý các luồng ưu tiên cao, nhạy độ trễ, khó song song hóa, hoặc những luồng giữ vai trò nút thắt của toàn ứng dụng. **C Core** tiếp nhận phần khối lượng còn lại, đặc biệt là các công việc có thể chia theo nhiều luồng, các tác vụ nền, các worker, các pipeline song song hoặc những phần việc mà tổng khối lượng quan trọng hơn đỉnh tức thời của một thread đơn.

Nhờ cách phân vai này, hệ thống đạt được hai điều cùng lúc. Thứ nhất, nó vẫn có những lõi cực mạnh cho các tình huống cần phản hồi tức thời và hiệu năng đơn luồng cao. Thứ hai, nó có thể mở rộng rất mạnh về tổng năng lực xử lý bằng cách triển khai thêm nhiều C Core. Chính vì vậy, nói rằng **C Core gia tăng sức mạnh đa nhân vì P Core đã gánh toàn bộ phần sức mạnh đơn nhân** là một cách diễn đạt hoàn toàn chính xác với tinh thần của sơ đồ.

Cái hay của C Core là nó không phải một phần phụ yếu ớt, mà là một “xương sống song song” của hệ thống. Khi workload phù hợp, lợi ích mà cả cụm C Core mang lại có thể vượt xa việc chỉ cố nâng thêm vài phần trăm hiệu năng đơn nhân trên P Core.

15. Những workload phù hợp nhất với C Core

Từ cấu trúc thể hiện trên sơ đồ, có thể thấy C Core đặc biệt phù hợp với các workload có thể phân rã thành nhiều thread hoặc nhiều công việc độc lập. Những ví dụ điển hình gồm biên dịch mã nguồn, render, encode và transcode media, xử lý ảnh theo lô, nén dữ liệu, phân tích dữ liệu, mô phỏng, tác vụ backend, dịch vụ nền, máy ảo, container, và các hệ thống có nhiều tiến trình chạy đồng thời.

Nó cũng phù hợp với những môi trường mà hệ điều hành cần xử lý liên tục nhiều hoạt động song song: dịch vụ nền, task scheduler, đồng bộ dữ liệu, indexing, telemetry, các thread hỗ trợ cho ứng dụng chính, hay những tác vụ không cần mức ưu tiên cao nhất nhưng vẫn cần năng lực xử lý thực chất. Với những bài toán như vậy, việc có nhiều C Core sẽ hiệu quả hơn nhiều so với việc dồn mọi gánh nặng lên một số ít P Core.

Ngoài ra, do có cụm vector/FP đủ mạnh và hệ bộ nhớ khá chắc, C Core còn phù hợp với những tác vụ dữ liệu vừa có thể chia theo thread, vừa có thể tận dụng SIMD. Đây là loại workload mà kiến trúc này có thể phát huy sức mạnh tổng hợp rất tốt, bởi mỗi lõi vừa xử lý được một phần dữ liệu, vừa bản thân từng lõi còn có năng lực vector hóa đáng kể.

16. Kết luận

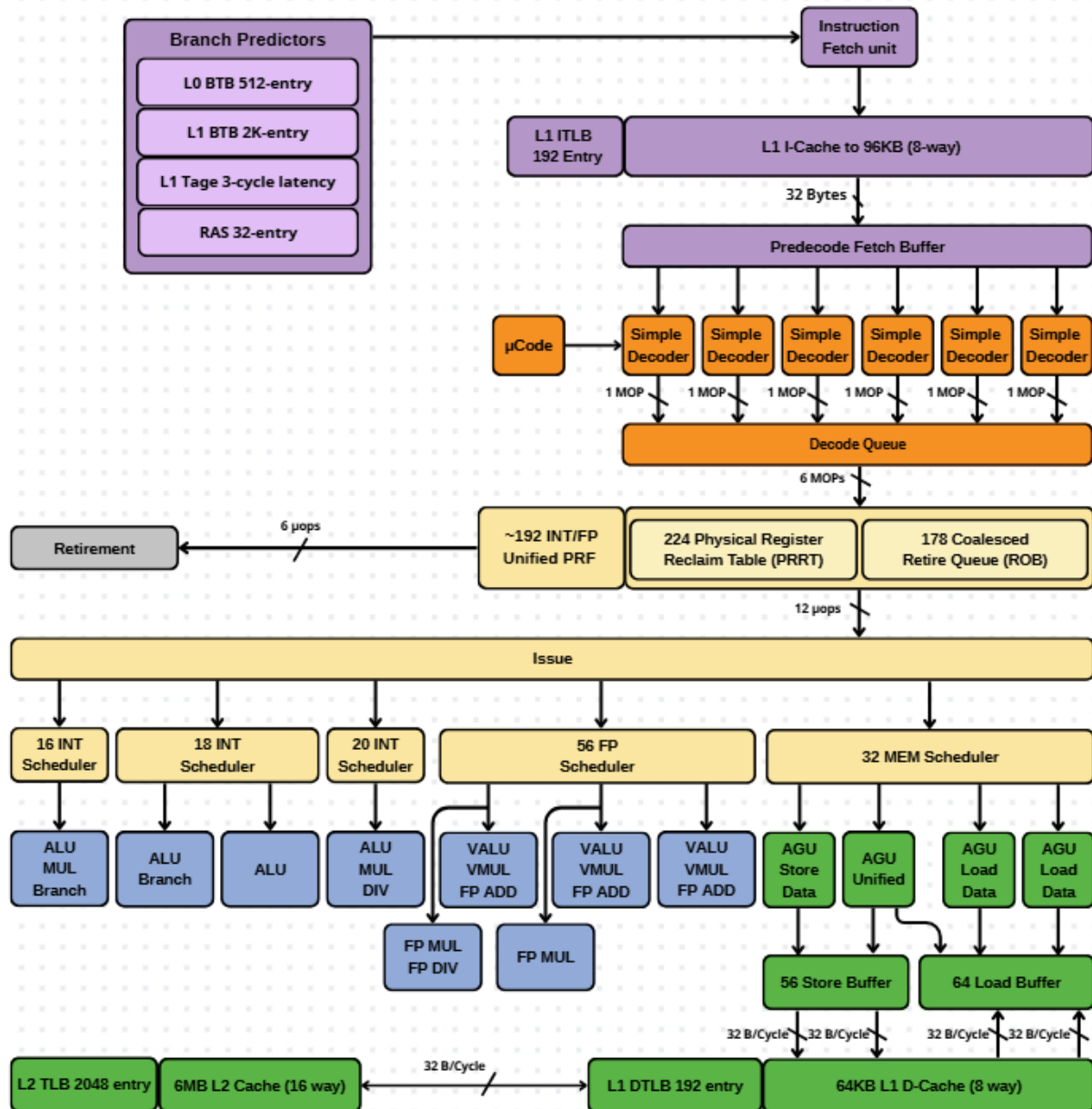
Sơ đồ **C Core** mô tả một kiến trúc CPU hiện đại, rộng và có chiều sâu, nhưng điểm cốt lõi của nó không nằm ở việc thay thế vai trò của **P Core** trong cuộc đua hiệu năng đơn nhân. Bản chất của thiết kế này là tối ưu cho **đa luồng, đa nhân, thông lượng tổng thể, và khả năng mở rộng sức mạnh xử lý của toàn con chip**.

Front-end của C Core đủ mạnh với predictor nhiều tầng, I-cache lớn, fetch rộng, predecode buffer, loop buffer và cụm decode 8 đơn vị. Phần giữa pipeline được đầu tư bằng hệ rename, PRF, PRRT và ROB đủ sâu để duy trì thực thi ngoài thứ tự hiệu quả. Back-end có issue rộng, retire rộng, nhiều scheduler phân cụm, nhiều execution unit integer, vector và FP. Hệ bộ nhớ lại đặc biệt đáng giá với AGU phong phú, load/store buffer lớn, TLB nhiều tầng, cache dữ liệu gần, và đặc biệt là **8MB Shared L2 Cache**, rất phù hợp với môi trường nhiều lõi cùng hoạt động.

Vì vậy, nếu phải tóm lược C Core trong một nhận định ngắn mà chính xác, có thể nói như sau: **P Core gánh đỉnh sức mạnh đơn nhân, còn C Core gánh sự mở rộng của sức mạnh tổng thể**. Giá trị của C Core không nằm ở việc một nhân đơn lẻ phải mạnh nhất, mà nằm ở việc **nhiều nhân như vậy cùng tồn tại sẽ tạo ra một nền hiệu năng đa nhân rất lớn, rất thực dụng, và rất phù hợp với các workload hiện đại vốn ngày càng thiên về song song hóa**.

E Core

E Core



1. Tổng thể kiến trúc và triết lý thiết kế

Nếu viết theo đúng bố cục phân tích ở trên, thì **E Core** trong sơ đồ này cho thấy một hướng thiết kế rất rõ: đây là một nhân được tối ưu cho **hiệu năng trên điện năng, mật độ tính toán cao**, và đặc biệt là **khả năng nhân rộng theo nhiều core** để nâng throughput toàn hệ thống. Nó không đi theo kiểu “siêu core” tập trung dồn thật nhiều tài nguyên vào đơn luồng,

mà chọn cách cân bằng hơn: front-end đủ rộng, back-end đủ sâu, hệ thống nhớ đủ mạnh để khi ghép nhiều E Core lại, tổng thông lượng đạt mức rất cao.

Điểm quan trọng là E Core ở đây không phải kiểu nhân nhỏ, hẹp, chỉ phù hợp việc nền. Ngược lại, nhìn vào predictor, decode width, ROB, scheduler và cụm FP/vector, có thể thấy đây vẫn là một **out-of-order core tương đối nghiêm túc**, chỉ khác ở chỗ mọi thứ được tối ưu theo hướng hợp lý hơn về diện tích và điện năng. Vì vậy, nếu đặt trong một kiến trúc lai hoặc một cụm nhiều nhân, E Core sẽ đóng vai trò nâng **throughput tổng**, xử lý tốt các workload song song, đa luồng, nền hệ thống, hoặc những tác vụ scale-out.

2. Front-end: branch prediction, fetch và decode

Phần front-end của E Core được xây khá đầy đủ, cho thấy Intel không coi đây là một khối fetch/decode tối giản. Cụm **Branch Predictors** gồm **L0 BTB 512-entry**, **L1 BTB 2K-entry**, **L1 TAGE 3-cycle latency** và **RAS 32-entry**. Tổ hợp này nói lên rằng E Core vẫn được ưu tiên mạnh về khả năng giữ pipeline ổn định khi gặp mã nhánh dày đặc, gọi hàm nhiều, hoặc luồng điều khiển phức tạp. Trong môi trường nhiều nhân, predictor tốt đặc biệt quan trọng vì sai nhánh không chỉ làm chậm một core mà còn làm giảm hiệu quả sử dụng tài nguyên trên quy mô cả cụm.

Phần lấy lệnh tiếp theo cũng khá thoáng với **Instruction Fetch Unit**, **L1 ITLB 192 entry** và **L1 I-Cache 96KB, 8-way**. Một **L1 I-cache 96KB** là kích thước đáng chú ý đối với E Core, cho thấy thiết kế cố tình giảm áp lực miss phía instruction-side để duy trì dòng lệnh đều hơn. Dòng fetch đi xuống **Predecode Fetch Buffer** ở mức **32 bytes**, nghĩa là front-end có năng lực nạp và chuẩn bị lệnh ở mức đủ rộng để không nhanh chóng trở thành nút thắt cổ chai.

Khối decode được tổ chức thành **6 simple decoders**, mỗi bộ giải mã **1 MOP**, cộng thêm đường **uCode** cho các trường hợp phức tạp, rồi đổ vào **Decode Queue 6 MOPs**. Đây là một cách thiết kế rất “đúng chất E Core”: vẫn duy trì decode rộng để đạt throughput tốt, nhưng dùng nhiều simple decoder thay vì đầu tư quá nặng vào decoder phức tạp. Kết quả là core xử lý hiệu quả các lệnh phổ biến trong thực tế với chi phí transistor và điện năng tốt hơn, rất phù hợp khi cần triển khai số lượng lớn nhân trên cùng một die.

3. Rename, ROB và cửa sổ out-of-order

Sau decode, sơ đồ cho thấy E Core có một tầng đổi tên và theo dõi lệnh đang bay không hề nhỏ. Ở đây có **~192 INT/FP Unified PRF**, **224 Physical Register Reclaim Table (PRRT)** và **178 Coalesced Retire Queue (ROB)**. Đây là một tín hiệu rất mạnh cho thấy E Core không phải một nhân “nông” về OoO. Một ROB ở mức **178 entries** cho phép giữ khá nhiều micro-ops đang bay đồng thời, từ đó che giấu tốt hơn độ trễ của bộ nhớ, phép toán dài, và cả các phụ thuộc dữ liệu.

Việc dùng **Unified PRF** cho cả integer và floating-point cũng cho thấy triết lý tối ưu tài nguyên khá thực dụng: giảm phân mảnh, linh hoạt hơn trong phân bổ register vật lý, và phù hợp với nhiều kiểu workload khác nhau. Trong bối cảnh nhiều core, điều này giúp mỗi nhân duy trì hiệu suất khá ổn mà không cần phải mở rộng quá mức từng khối riêng lẻ. Nói cách khác, E Core không cố thắng tuyệt đối về IPC đơn luồng, nhưng có đủ cửa sổ OoO để tránh rơi vào trạng thái “kẹt” khi chạy mã thực tế.

Bảng **retirement 6 μops** và đường cấp xuống **issue 12 μops** cũng cho thấy một mức độ tham vọng đáng kể. Khả năng issue rộng hơn retirement là cách quen thuộc để giữ cho back-end luôn được nuôi đủ công việc, đặc biệt khi có stall cục bộ ở một vài đường thực thi. Điều này phù hợp với mục tiêu throughput: thay vì tối ưu cho latency thấp nhất của một chuỗi ngắn, thiết kế ưu tiên giữ thật nhiều công việc tiến triển song song trong pipeline.

4. Back-end integer: tài nguyên nguyên số và xử lý nhánh

Phần nguyên số của E Core được chia thành **ba cụm INT Scheduler** với kích thước **16**, **18** và **20 entries**. Cách chia cụm như vậy cho thấy back-end được phân tách để giảm tranh chấp tài nguyên và cho phép nhiều loại vi lệnh integer cùng tiến hành song song. Đây là điểm thường thấy ở những kiến trúc tối ưu cho hiệu quả thực tế: không cần một scheduler khổng lồ duy nhất, mà dùng các cụm hợp lý để cân bằng giữa độ rộng, độ phức tạp và diện năng.

Các execution units đi kèm gồm các khối như **ALU**, **ALU Branch**, **ALU MUL**, và **ALU MUL DIV**. Điều này cho thấy E Core có đủ đường để xử lý song song cả phép toán số nguyên cơ bản, nhánh, nhân và chia. Với các workload đa dụng như hệ điều hành, dịch mã, logic ứng dụng, web back-end, scripting, hoặc tác vụ nền, cụm nguyên số này là rất quan trọng vì phần lớn công việc thực tế vẫn xoay quanh integer, branch và address-related operations.

Một điểm đáng chú ý là E Core không hy sinh hoàn toàn năng lực branch/int để lấy mật độ. Ngược lại, nó vẫn đầu tư đủ sâu để mỗi core có thể đứng vững độc lập khi chạy thread riêng. Chính điều đó làm cho E Core trở thành một khối scale-out tốt: thay vì chỉ sống nhờ số lượng, từng core vẫn có năng lực đủ mạnh để đóng góp thật sự vào throughput chung.

5. Back-end FP/vector: không chỉ là core “nhẹ”

Khối dấu chấm động và vector của E Core cũng cho thấy đây không phải một nhân chỉ hợp tác vụ nền đơn giản. Sơ đồ thể hiện **56 FP Scheduler** cùng nhiều đơn vị dạng **VALU / VMUL / FP ADD**, kèm thêm các khối **FP MUL / FP DIV** chuyên dụng. Điều đó nghĩa là E Core có năng lực tương đối tốt với các workload có thành phần SIMD, vector và floating-point, chứ không chỉ dừng ở integer thông thường.

Về mặt kiến trúc, đây là quyết định rất hợp lý. Nếu một E Core chỉ mạnh ở integer cơ bản, giá trị của nó trong hệ thống hiện đại sẽ bị giới hạn. Nhưng với cụm FP/vector tương đối đầy đủ, nhiều E Core có thể cùng tham gia vào những workload như media, nén dữ liệu, xử lý tín hiệu, analytics, hoặc inference nhẹ. Khi đó, tổng hiệu năng đạt được trên mỗi watt sẽ rất ấn tượng, dù mỗi nhân riêng lẻ không được đẩy lên cực điểm như P Core.

Điều quan trọng là triết lý ở đây vẫn không phải “hy sinh hết để lấy diện tích nhỏ nhất”, mà là **chọn mức tài nguyên vừa đủ để mỗi core đa dụng hơn**, từ đó làm tăng hiệu quả thực chiến của cả cụm E Core. Đây là kiểu đầu tư rất đúng cho môi trường mà số luồng công việc đồng thời lớn hơn nhiều so với nhu cầu tối ưu một luồng đơn lẻ.

6. Memory subsystem: load/store đủ rộng để nuôi throughput

Một trong những phần ấn tượng nhất của E Core trong sơ đồ này là hệ thống load/store và memory scheduling. Ta thấy có **32 MEM Scheduler**, đi cùng **4 AGU** gồm **AGU Store Data**, **AGU Unified**, và **2 AGU Load Data**. Cấu hình này cho thấy E Core không chỉ mạnh về compute mà còn được thiết kế để xử lý lưu lượng bộ nhớ khá nghiêm túc, điều rất quan trọng trong workload thực tế nơi bottleneck thường nằm ở dữ liệu chứ không phải chỉ ở ALU.

Ngoài AGU, core còn có **56 Store Buffer** và **64 Load Buffer**. Đây là những con số khá lớn với một efficient core, cho phép theo dõi đồng thời nhiều thao tác bộ nhớ đang bay, từ đó khai thác tốt hơn **memory-level parallelism**. Khi một load bị trễ hoặc một store chưa thể

hoàn tất ngay, pipeline vẫn có không gian để tiếp tục đẩy các thao tác khác tiến lên. Trong bối cảnh đa nhân, điều này đặc biệt hữu ích vì nó giúp mỗi core giữ throughput tốt ngay cả khi working set dữ liệu không hoàn hảo hoặc truy cập bộ nhớ có độ phân tán cao.

Nhìn rộng hơn, cụm memory subsystem này cho thấy E Core được dựng để làm việc thật sự trong môi trường nhiều luồng và nhiều core, nơi tắc nghẽn bộ nhớ thường là giới hạn chính. Một core muốn scale tốt theo số lượng không thể chỉ có ALU mạnh; nó còn phải có đủ khả năng phát sinh, theo dõi và hấp thụ lưu lượng load/store. Sơ đồ này cho thấy E Core đáp ứng khá tốt yêu cầu đó.

7. Cache hierarchy và TLB: tối ưu locality cho cụm nhiều core

Phần cache và TLB tiếp tục củng cố nhận định rằng E Core được tối ưu cho throughput hệ thống. Ở phía dữ liệu, ta có **L1 DTLB 192 entry, 64KB L1 D-Cache (8-way), L2 TLB 2048 entry, và 6MB L2 Cache (16-way)**. Đây là một cấu hình rất đáng nể với một nhân thuộc nhóm efficient, đặc biệt là khi nhìn theo góc độ hỗ trợ nhiều core hoạt động đồng thời.

L1 D-cache 64KB giúp giảm miss ở cấp gần nhất, rất quan trọng với các workload có locality vừa phải nhưng không hoàn hảo. **L2 TLB 2048 entry** hỗ trợ tốt hơn cho các không gian địa chỉ lớn và khối lượng truy cập bộ nhớ dày đặc. Trong khi đó, **6MB L2 cache 16-way** là một tài nguyên rất giá trị để giảm áp lực lên các tầng nhớ xa hơn, giữ lại working set cỡ trung bình ngay gần cụm core hơn. Với nhiều E Core chạy song song, một L2 tương đối lớn như vậy có thể cải thiện rõ rệt throughput chung bằng cách giảm độ nhạy với DRAM latency và giảm tranh chấp băng thông ngoài chip.

Các đường truyền **32 B/cycle** giữa các khối dữ liệu và cache cho thấy kiến trúc cũng quan tâm rõ đến băng thông nội bộ chứ không chỉ dung lượng. Điều này quan trọng vì cache lớn mà feed dữ liệu không đủ nhanh thì execution units vẫn bị đói. Sơ đồ chỉ ra rằng E Core không đơn thuần có bộ nhớ đệm lớn, mà còn có khả năng luân chuyển dữ liệu ở mức hợp lý để nuôi các cụm thực thi.

8. Ý nghĩa hiệu năng: E Core mạnh ở đâu, yếu ở đâu

Từ toàn bộ sơ đồ, có thể rút ra rằng E Core sẽ đặc biệt mạnh trong những kịch bản như **nhiều luồng độc lập, throughput-oriented workloads, nhiệm vụ nền quy mô lớn**, và các môi trường mà **tổng công việc hoàn thành mỗi giây** quan trọng hơn độ trễ của một luồng

duy nhất. Nó phù hợp với kiểu tải như service song song, xử lý nền hệ thống, compile phân tán theo thread, data processing nhẹ đến trung bình, hoặc các tác vụ vector/FP không đòi hỏi tài nguyên cực đại trên từng core.

Ngược lại, nếu mục tiêu là đạt đỉnh cao nhất cho **single-thread latency**, E Core về bản chất vẫn không đi theo hướng đó. Dù front-end, OoO window và back-end đều khá tốt, thiết kế tổng thể vẫn thiên về cân bằng chi phí hơn là tối đa hóa mọi chỉ số IPC đơn luồng. Vì thế, khi so với một P Core hay một core lớn thiên về burst performance, E Core có thể thua ở đỉnh hiệu năng từng luồng. Nhưng đổi lại, nó thắng lớn ở **mật độ hiệu năng, hiệu suất năng lượng**, và **khả năng nhân rộng số lượng**.

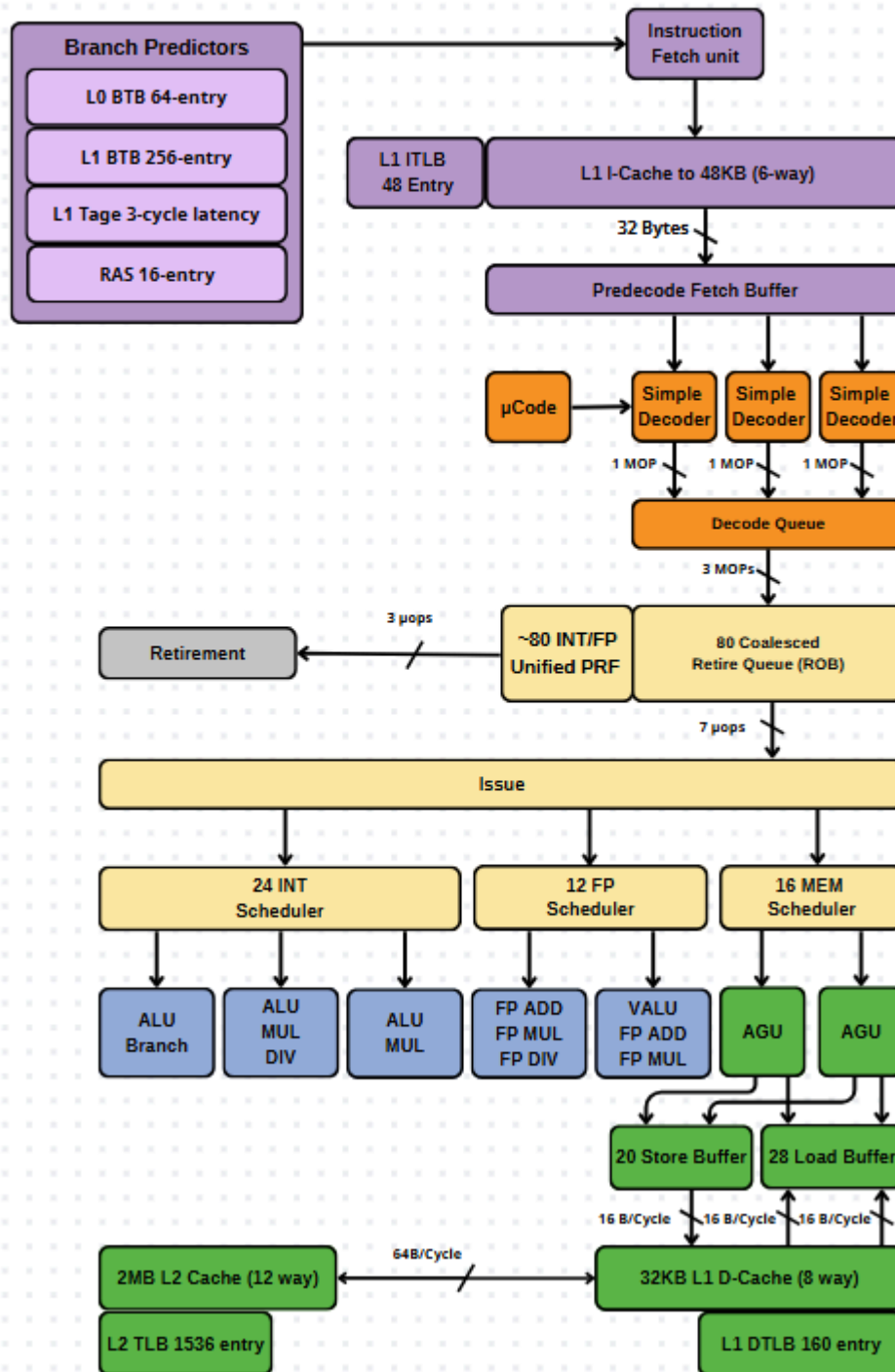
9. Kết luận

Nhìn theo đúng bố cục phân tích phía trên, **E Core** trong sơ đồ này là một nhân được thiết kế rất bài bản cho triết lý **scale-out**. Front-end đủ rộng để không nghẽn sớm, predictor đủ tốt để giữ pipeline ổn định, cửa sổ OoO đủ sâu để che giấu độ trễ, execution resources đủ đa dạng để chạy nhiều kiểu workload, và memory/cache subsystem đủ mạnh để sustain throughput khi tải thực tế tăng lên.

Tóm lại, E Core ở đây không phải “core nhỏ yếu”, mà là một **nhân hiệu quả cao, cân bằng tốt và rất phù hợp cho kiến trúc nhiều core**. Giá trị lớn nhất của nó không nằm ở việc thắng một luồng đơn lẻ, mà nằm ở chỗ khi đặt nhiều E Core cạnh nhau, hệ thống có thể đạt

LPE core

LPE Core



1. Tổng thể kiến trúc và vai trò thiết kế

Với **LPE Core**, cách đọc hợp lý nhất theo đúng bố cục ở trên là: đây là một nhân được tối ưu rất rõ cho vai trò **xử lý nền, tải nhẹ, tác vụ duy trì hệ thống và các luồng không cần hiệu năng bùng nổ**, từ đó **giải phóng tài nguyên** cho các nhân mạnh hơn như **P Core, C Core và E Core** tập trung vào phần việc nặng hơn. Nói cách khác, LPE Core không được xây để cạnh tranh đỉnh hiệu năng đơn luồng, mà để đảm nhận những công việc “luôn phải có người làm”, nhưng không đáng để gọi tới các core lớn và tốn điện hơn.

Nhìn tổng thể sơ đồ, có thể thấy triết lý của LPE Core là **thu gọn hợp lý mọi khối quan trọng của một core hiện đại**: vẫn có branch predictor riêng, vẫn có fetch/decode tách lớp, vẫn có **out-of-order engine**, vẫn có cụm integer, FP, memory riêng, cùng hệ thống cache/TLB đầy đủ. Điều đó cho thấy đây không phải chỉ là một nhân cực nhỏ kiểu tối giản, mà là một core hoàn chỉnh nhưng được “nén” lại để tối ưu **điện năng, diện tích và khả năng duy trì hoạt động liên tục với chi phí thấp**.

Nếu P Core, C Core hay E Core hướng nhiều hơn tới throughput hoặc hiệu năng cao, thì **LPE Core giống một lớp xử lý nền thường trực**: đủ thông minh để chạy nhiều loại công việc thật, nhưng đủ nhỏ và tiết kiệm để không làm tăng chi phí năng lượng toàn hệ thống khi chỉ cần xử lý tác vụ nhẹ.

2. Front-end: branch prediction, fetch và decode

Phần front-end của LPE Core cho thấy đây là một thiết kế tiết chế hơn đáng kể so với E Core, đúng với vai trò nền của nó. Cụm branch predictor gồm **L0 BTB 64-entry, L1 BTB 256-entry, L1 TAGE 3-cycle latency** và **RAS 16-entry**. Các con số này nhỏ hơn rõ rệt, cho thấy predictor được xây ở mức **đủ dùng cho workload nền**, thay vì tối ưu để theo đuổi khả năng dự đoán cực mạnh trên mã nhánh phức tạp hoặc working set lệnh lớn.

Dù vậy, việc vẫn giữ đầy đủ các thành phần như **BTB nhiều tầng, TAGE và RAS** rất có ý nghĩa. Nó cho thấy LPE Core không phải kiểu in-order cực đơn giản, mà vẫn được đầu tư để giữ luồng lệnh ổn định, tránh lãng phí pipeline ngay cả khi chạy các tác vụ nền có điều kiện rẽ nhánh, gọi hàm, vòng lặp và logic điều khiển phổ biến. Với một core chuyên xử lý công việc thường trực của hệ thống, tính ổn định này quan trọng hơn việc mở predictor thật lớn.

Ở tầng lấy lệnh, sơ đồ cho thấy **L1 ITLB 48 entry** và **L1 I-Cache 48KB (6-way)**, đi kèm luồng **32 bytes** xuống **Predecode Fetch Buffer**. So với các core lớn hơn, đây là cấu hình gọn hơn, nhưng hoàn toàn hợp lý với mục tiêu của LPE: các tác vụ nền thường có code footprint vừa phải, lặp đi lặp lại nhiều, nên không cần instruction-side quá lớn. Thiết kế như vậy giúp tiết kiệm diện tích mà vẫn đảm bảo core có đủ không gian để chạy mượt các luồng điều phối, dịch vụ nền, quản lý hệ thống, xử lý tín hiệu nhẹ hoặc các tác vụ housekeeping.

Khối decode gồm **3 simple decoders**, mỗi bộ **1 MOP**, kèm **uCode**, rồi đi vào **Decode Queue 3 MOPs**. Đây là một front-end hợp hơn rõ rệt, phản ánh đúng vai trò của LPE Core: không nhắm tới việc hút thật nhiều lệnh mỗi chu kỳ, mà nhắm tới **xử lý ổn định, tiết kiệm và đủ năng lực cho tác vụ nhẹ**. Việc dùng simple decoder thay vì tổ hợp giải mã quá phức tạp cũng là quyết định hợp lý nếu mục tiêu chính là **efficiency-first**.

3. Rename, ROB và chiều sâu out-of-order

Điểm rất đáng chú ý là dù nhỏ hơn, LPE Core **vẫn là một core out-of-order đúng nghĩa**. Sơ đồ cho thấy nó có **~80 INT/FP Unified PRF** và **80 Coalesced Retire Queue (ROB)**. Điều này cực kỳ quan trọng, vì nó cho thấy LPE Core không chỉ đơn giản chạy tuần tự từng lệnh như một nhân siêu tối giản, mà vẫn có khả năng giữ một cửa sổ lệnh đang bay đủ để che giấu độ trễ cục bộ và duy trì hiệu suất tốt hơn trên workload thực.

Một **ROB khoảng 80 entries** không lớn nếu so với E Core hay các core hiệu năng cao hơn, nhưng với vai trò của LPE Core thì đây là mức rất hợp lý. Nó đủ để khai thác một phần song song mức lệnh, xử lý phụ thuộc, và hấp thụ một số trễ bộ nhớ hay trễ thực thi mà không cần mở rộng quá mạnh phần rename/retire. Nói cách khác, LPE Core vẫn “thông minh”, chỉ là mức thông minh đó được đặt trong một giới hạn tối ưu hơn cho điện năng.

Bảng thực thi cũng phản ánh triết lý này. Từ sơ đồ có thể thấy đường đi xuống **issue 7 μops** và **retirement 3 μops**. Điều này cho thấy LPE Core vẫn có mức issue tương đối linh hoạt so với quy mô tổng thể, nhưng retire được giữ ở mức khiêm tốn. Đây là dấu hiệu quen thuộc của một core nền: **đủ rộng để không quá dễ nghẽn**, nhưng không mở lớn tới mức làm tăng độ phức tạp và điện năng một cách không cần thiết.

4. Back-end integer: tối ưu cho điều phối và xử lý nền

Khối integer của LPE Core được gom vào **24 INT Scheduler**, đi kèm các đơn vị **ALU Branch**, **ALU MUL DIV**, và **ALU MUL**. Cách tổ chức này cho thấy LPE Core vẫn được ưu tiên cho những kiểu tác vụ vốn rất phổ biến ở công việc nền: điều kiện rẽ nhánh, tính toán số nguyên cơ bản, thao tác địa chỉ, logic điều phối, quản lý trạng thái và các phép toán số học không quá nặng.

Điểm cần nhấn mạnh là với một core chạy nền, integer side thường quan trọng hơn nhiều so với việc theo đuổi vector cực mạnh. Hầu hết tải nền của hệ điều hành và ứng dụng đều thiên về **control flow, integer logic, quản lý tài nguyên, interrupt, timer, scheduling, housekeeping**, và các tác vụ “điều hành” hơn là các kernel tính toán lớn. Cấu hình integer của LPE Core vì thế rất hợp vai: không phô trương, nhưng đủ để gánh phần việc thường nhật một cách độc lập và liên tục.

Việc có riêng khối **ALU Branch** cũng cho thấy ngay cả ở một core tiết kiệm điện, xử lý nhánh vẫn được xem trọng. Điều đó hợp lý, vì nhiều tác vụ nền tuy không nặng compute nhưng lại có mật độ nhánh khá cao. Một core chạy nền tốt không cần phải cực rộng, nhưng cần **điều khiển luồng lệnh mượt và ít lãng phí**.

5. Back-end FP/vector: có nhưng ở mức thực dụng

Phần dấu chấm động và vector của LPE Core nhỏ gọn hơn rõ rệt, với **12 FP Scheduler** và hai cụm chính: một cụm dạng **FP ADD / FP MUL / FP DIV**, và một cụm dạng **VALU / FP ADD / FP MUL**. Điều này cho thấy LPE Core vẫn hỗ trợ tương đối đầy đủ các loại tác vụ FP/vector cơ bản, nhưng không đặt đây là trọng tâm thiết kế.

Đây là lựa chọn rất hợp lý. Một core nền không thể hoàn toàn thiếu FP/vector vì workload thực tế vẫn có thể đụng tới các phép tính media nhẹ, xử lý tín hiệu nhỏ, chuyển đổi dữ liệu, hoặc các thư viện nền có dùng toán FP. Nhưng vì vai trò chính của LPE Core không phải xử lý compute-heavy hay data-parallel nặng, nên tài nguyên FP được giữ ở mức **đủ linh hoạt**, thay vì mở rộng như trên E Core hoặc các core lớn hơn.

Nói cách khác, LPE Core vẫn là một core đa dụng, chứ không bị giới hạn vào một loại công việc quá hẹp. Tuy nhiên, toàn bộ cụm FP/vector cho thấy rõ rằng mục tiêu của nó là **bao**

phủ nhu cầu nền và tác vụ nhẹ, chứ không phải trở thành khối tính toán song song chủ lực của hệ thống.

6. Memory subsystem: đủ để xử lý tải nền mà không nghẽn sớm

Khối bộ nhớ của LPE Core cũng được thiết kế đúng tinh thần “gọn nhưng không sơ sài”. Sơ đồ thể hiện **16 MEM Scheduler**, **2 AGU**, cùng **20 Store Buffer** và **28 Load Buffer**. So với E Core, đây là quy mô nhỏ hơn đáng kể, nhưng vẫn đủ để cho thấy LPE Core không bị bó hẹp tới mức rất dễ nghẽn khi workload có hoạt động load/store liên tục.

Hai **AGU** là dấu hiệu rằng core vẫn có khả năng xử lý song song các thao tác địa chỉ ở mức cơ bản, phù hợp với những tác vụ nền như copy dữ liệu nhỏ, quản lý bộ đệm, polling, cập nhật trạng thái, hoặc các luồng hệ thống có nhiều truy cập bộ nhớ không quá nặng. **Load Buffer 28** và **Store Buffer 20** cũng cho phép core giữ một lượng thao tác nhớ đang bay đủ để hấp thụ trễ cục bộ mà không làm pipeline đứng lại ngay.

Điểm này rất quan trọng nếu đặt LPE Core vào đúng vai trò mà bạn nêu: **gánh việc nền để giải phóng P Core, C Core và E Core**. Muốn làm được điều đó, nó không thể chỉ chạy tốt những đoạn code hoàn hảo nằm gọn trong register; nó còn phải xử lý được hoạt động bộ nhớ đời thường của hệ điều hành và service nền. Sơ đồ cho thấy nó được trang bị vừa đủ để làm điều đó một cách kinh tế.

7. Cache hierarchy và TLB: tối ưu cho footprint nhỏ, hoạt động liên tục

Hệ thống cache và dịch địa chỉ của LPE Core tiếp tục phản ánh vai trò chuyên xử lý nền. Ở đây ta thấy **32KB L1 D-Cache (8-way)**, **L1 DTLB 160 entry**, **L2 TLB 1536 entry**, và **2MB L2 Cache (12-way)**. Đây là một cấu hình nhỏ hơn so với các core lớn hơn, nhưng hoàn toàn hợp lý với một core vốn được kỳ vọng chạy các workload có **working set vừa hoặc nhỏ**, lặp lại nhiều, và ít đòi hỏi dung lượng cache cực lớn.

L1 D-cache 32KB là mức vừa đủ cho nhiều tác vụ nền có locality ngắn hạn. **L1 DTLB 160 entry** và **L2 TLB 1536 entry** cho thấy phần dịch địa chỉ vẫn được chăm chút, điều rất quan trọng vì workload hệ thống có thể chạm nhiều vùng nhớ nhỏ, nhiều ngữ cảnh và nhiều cấu

trúc dữ liệu phân tán. Một LPE Core nếu bị yếu ở TLB thì rất dễ mất lợi thế hiệu quả khi chạy tác vụ nền thật.

Phần **2MB L2 Cache (12-way)** là điểm khá đáng giá. Với một core nền, 2MB L2 cho thấy thiết kế không quá keo kiệt; nó vẫn cho core đủ không gian để giữ dữ liệu gần, giảm truy cập ra xa hơn, và duy trì hoạt động ổn định mà không cần đánh thức các core lớn chỉ để xử lý những việc nhỏ nhưng kéo dài. Đường truyền **64 B/cycle** giữa L1D và L2 cũng cho thấy dữ liệu được luân chuyển ở mức đủ khỏe để không làm memory-side trở thành nút thắt quá sớm.

8. Ý nghĩa hiệu năng và vai trò trong hệ thống lai

Nếu đặt LPE Core vào bối cảnh hệ thống có **P Core, C Core và E Core**, thì vai trò của nó rất dễ hiểu: đây là lớp core chuyên hấp thụ các công việc **nền, thường trực, nhạy điện năng nhưng không nhạy hiệu năng đỉnh**. Những việc như duy trì dịch vụ nền, tác vụ quản lý hệ thống, xử lý sự kiện nhẹ, housekeeping, polling, background agents, hoặc các luồng phụ trợ có thể được đẩy sang LPE Core để các core mạnh hơn tập trung cho phần việc thực sự cần nhiều tài nguyên.

Lợi ích lớn nhất của LPE Core vì thế không nằm ở việc “mạnh riêng nó”, mà ở chỗ nó **làm giảm lãng phí toàn hệ thống**. Nếu một tác vụ nền nhỏ vẫn chạy trên P Core hay các core lớn hơn, hệ thống sẽ tiêu tốn nhiều điện năng và chiếm mất tài nguyên quý cho phần việc không xứng đáng. LPE Core giải quyết bài toán đó bằng cách cung cấp một nơi thực thi rẻ hơn nhưng vẫn đủ năng lực để không làm các luồng nền trở nên quá chậm.

Nói ngắn gọn, nếu P/C/E là các tầng xử lý hướng tới hiệu năng và throughput, thì **LPE là tầng “neo nền”** giúp hệ thống duy trì trạng thái hoạt động thông minh hơn về điện năng. Chính sự hiện diện của nó có thể giúp toàn bộ cấu trúc di thể hoạt động hiệu quả hơn, không chỉ vì thêm một core nữa, mà vì **đặt đúng loại việc vào đúng loại core**.

9. Kết luận

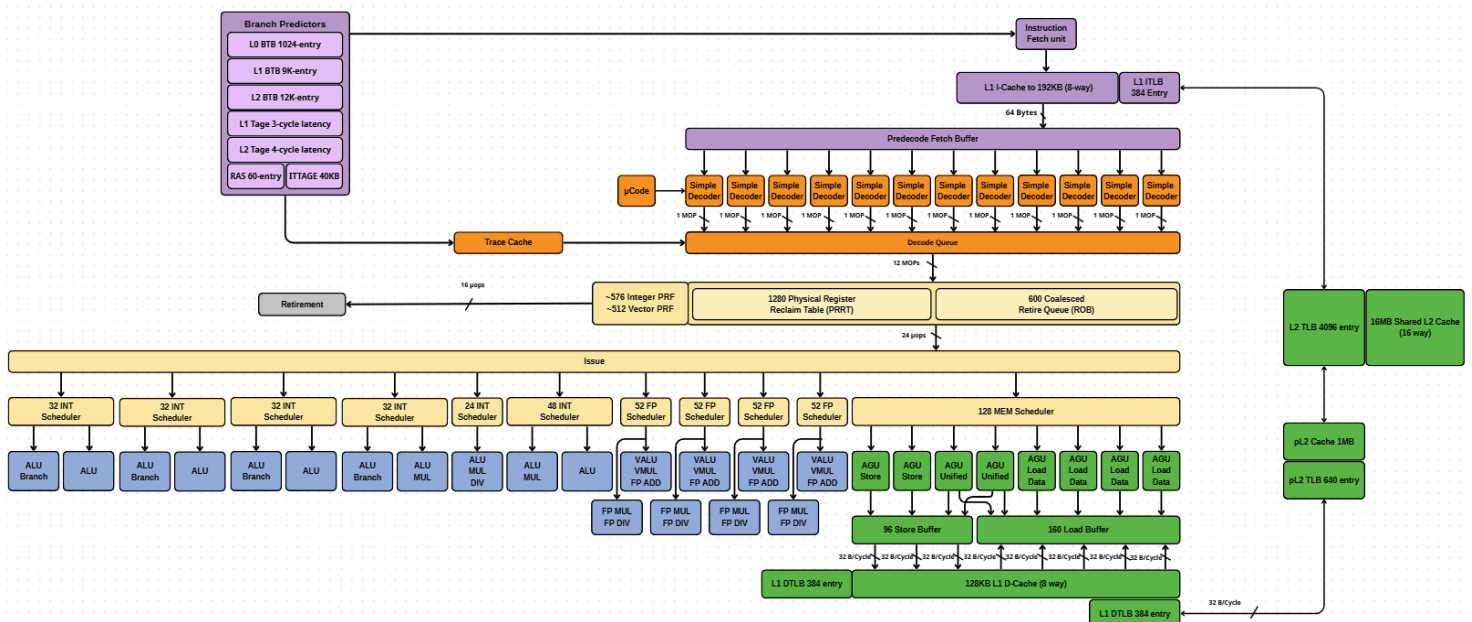
Theo đúng bố cục phân tích ở trên, **LPE Core** là một nhân được thiết kế rất có chủ đích cho vai trò **background CPU**. Nó có front-end gọn hơn, predictor nhỏ hơn, decode hẹp hơn, ROB và PRF nhỏ hơn, execution resources vừa phải hơn, cùng cache/memory subsystem cũng được co lại so với E Core hay các core lớn hơn. Nhưng điểm quan trọng là sự thu gọn

này không làm nó trở thành một core “thô sơ”; ngược lại, nó vẫn là một **core out-of-order hoàn chỉnh**, đủ sức xử lý đa dạng tác vụ nền một cách độc lập và hiệu quả.

Vì vậy, cách hiểu đúng nhất về LPE Core là: **một nhân tối ưu cho điện năng và tải nền, được sinh ra để gánh các công việc duy trì hệ thống và giải phóng những core hiệu suất cao cho các tác vụ quan trọng hơn**. Giá trị của nó không nằm ở đỉnh hiệu năng đơn luồng, mà ở **khả năng giữ cho toàn hệ thống vận hành hiệu quả hơn, mát hơn, và tiết kiệm hơn**.

ARC core

ARC Core



1. Tổng thể kiến trúc và vai trò của ARC Core

Nếu viết theo đúng kiểu phân tích dành cho một **core chủ lực** như P Core, thì **ARC Core** trong sơ đồ này hiện lên như một nhân **general-purpose rất lớn, rất rộng, và rất “đắt tài nguyên”**, được dựng lên để đảm nhiệm phần việc điều phối trung tâm của hệ thống. Khác với một khối tính toán chuyên biệt chỉ tối ưu cho một dạng workload hẹp, ARC Core có đủ những dấu hiệu của một **core điều khiển cấp cao**: front-end cực mạnh, cửa sổ out-of-order rất sâu, tài nguyên integer và memory rất dày, cùng hệ thống cache/TLB nhiều tầng có quy mô lớn. Chỉ nhìn vào cách phân bổ transistor giữa predictor, fetch, decode, rename, issue,

execution và cache, có thể thấy đây không phải một core “phụ trợ”, mà là một **bộ não điều hành tính toán tổng quát**.

Trong bối cảnh bạn đặt ra rằng **ARC Core sẽ quản lý cả cụm SPE**, vai trò kiến trúc của nó càng trở nên rõ ràng hơn. Một cụm SPE, xét từ sơ đồ riêng của SPE, thiên về **xử lý song song đậm đặc, vector hóa mạnh, local store riêng, DMA và pipeline chuyên dụng**. Một khối như vậy rất phù hợp để “cày” dữ liệu, nhưng không phải nơi lý tưởng cho những công việc như **điều phối luồng tác vụ, phân mảnh công việc, xây dựng command stream, xử lý logic nhánh phức tạp, quản lý phụ thuộc, đồng bộ hóa, giao tiếp với bộ nhớ chia sẻ, hay hấp thụ các đoạn mã tuần tự và điều khiển**. ARC Core chính là phần kiến trúc bù vào khoảng trống đó.

Vì vậy, nếu nhìn toàn hệ thống theo tinh thần host–accelerator, thì **ARC Core đóng vai trò host core điều khiển**, còn **SPE là cụm engine tăng tốc**. Giá trị lớn nhất của ARC không nằm ở chỗ nó chỉ “mạnh” theo nghĩa chung chung, mà ở chỗ nó sở hữu đúng những phẩm chất mà một core điều phối SPE cần có: **khả năng chạy mã điều khiển rất tốt, xử lý được nhánh dày đặc, chịu được độ trễ bộ nhớ, theo dõi lượng lớn trạng thái đang bay, và duy trì thông lượng quản trị cao ngay cả khi toàn cụm tăng tốc đang hoạt động nặng**.

2. Front-end: một khối lấy lệnh và dự đoán nhánh được xây theo tiêu chuẩn rất cao

Điểm đầu tiên khiến ARC Core nổi bật chính là phần **branch prediction**. Sơ đồ cho thấy cụm predictor của nó bao gồm **L0 BTB 1024-entry, L1 BTB 8K-entry, L2 BTB 12K-entry, L1 TAGE 3-cycle latency, L2 TAGE 4-cycle latency, RAS 60-entry và ITTAGE 40KB**. Đây là một tổ hợp rất giàu tài nguyên, và chỉ riêng chi tiết này thôi cũng đã nói lên nhiều điều về triết lý thiết kế của ARC.

Một core được giao vai trò quản lý SPE không thể yếu ở điều khiển luồng. Phần mềm điều phối tăng tốc thường đầy rẫy **nhánh, vòng lặp, bảng điều phối, hàng đợi lệnh, kiểm tra trạng thái, xử lý bất đồng bộ, phân tích điều kiện tài nguyên, dựng descriptor DMA, thiết lập barrier và quản lý ngoại lệ**. Những đoạn mã kiểu đó không “đẹp” như kernel số học tuyến tính; chúng có tính điều khiển cao, nhiều đường đi, nhiều phụ thuộc động. Một predictor nghèo tài nguyên sẽ khiến pipeline vỡ nhịp liên tục. Ngược lại, predictor rất mạnh

như trên ARC cho thấy core này được thiết kế để giữ **single-thread responsiveness** ở mức cao ngay cả khi mã điều khiển phức tạp.

Phía sau predictor là **Instruction Fetch Unit, L1 I-Cache 192KB (8-way), L1 ITLB 384 entry** và luồng fetch **64 bytes** xuống **Predecode Fetch Buffer**. Một **L1 I-cache 192KB** là cực kỳ đáng chú ý. Đây là dấu hiệu của một front-end được đầu tư mạnh để hấp thụ **instruction footprint lớn**, rất phù hợp với các runtime phức tạp, mã hệ thống, thư viện điều phối, cũng như các đoạn code phối hợp nhiều lớp chức năng. **L1 ITLB 384 entry** cũng giúp giảm áp lực dịch địa chỉ phía instruction-side, điều đặc biệt quan trọng nếu ARC là nơi chạy lớp phần mềm quản trị, scheduler, driver-like logic hoặc runtime điều khiển cụm SPE.

Sự xuất hiện của **Trace Cache** là một điểm rất “đắt giá” về mặt kiến trúc. Nó gợi ý rằng ARC không chỉ muốn decode rộng, mà còn muốn **tái sử dụng các đường lệnh đã được xử lý trước** để giảm áp lực lên front-end và duy trì feed-rate ổn định cho back-end. Với một core host quản lý accelerator, đây là lựa chọn rất hợp lý: nhiều vòng điều phối, dispatch loop, polling loop, completion handling loop hay scheduling path thường lặp lại rất nhiều. Trace cache giúp những đoạn điều khiển “nóng” đó quay lại pipeline nhanh hơn, đều hơn, ít tốn năng lượng front-end hơn.

Khối decode tiếp tục củng cố nhận định này. Sơ đồ cho thấy **12 simple decoders**, mỗi bộ **1 MOP**, có đường **uCode**, rồi đổ vào **Decode Queue 12 MOPs**. Một bề rộng decode như vậy là rất lớn. Nó cho phép ARC giữ tốc độ nạp vi lệnh cao, rất hữu ích khi phải hấp thụ lượng mã điều khiển lớn, xử lý các đoạn setup phức tạp trước khi phát công việc xuống cụm SPE, và vẫn còn đủ headroom để chạy những phần mã hỗn hợp có cả scalar, vector và memory operations. Đây không phải front-end của một core “vừa đủ dùng”; đây là front-end của một core được kỳ vọng phải **đứng ở tuyến đầu xử lý logic tổng quát**.

3. Rename, PRF, ROB và cửa sổ out-of-order: dấu hiệu của một core cực sâu

Nếu front-end cho thấy ARC rất mạnh ở việc *đưa lệnh vào máy*, thì tầng rename và theo dõi vi lệnh đang bay cho thấy nó còn mạnh ở việc **giữ một lượng công việc khổng lồ cùng tiến triển đồng thời**. Sơ đồ thể hiện **~576 Integer PRF**, **~512 Vector PRF**, **1280 Physical Register Reclaim Table (PRRT)** và **600 Coalesced Retire Queue (ROB)**. Đây là những

con số của một thiết kế out-of-order rất sâu, thậm chí thuộc loại “khổng lồ” nếu đặt trong bối cảnh một core điều khiển tổng quát.

Một **ROB 600 entries** có ý nghĩa rất lớn. Nó cho phép ARC giữ lại một cửa sổ lệnh đang bay cực rộng, nhờ đó core có thể **che giấu độ trễ bộ nhớ**, chịu đựng được các đoạn mã có nhiều phụ thuộc không đều, duy trì tiến độ ngay cả khi một số nhánh hoặc thao tác load/store bị chậm, đồng thời tiếp tục tranh thủ thực thi những phần độc lập khác ở phía sau. Đây là phẩm chất đặc biệt quan trọng đối với một core quản lý SPE, bởi phần việc của nó thường không phải là chuỗi số học lý tưởng, mà là **một hỗn hợp phức tạp giữa điều khiển, địa chỉ hóa, truy cập cấu trúc dữ liệu, đồng bộ, phân phối công việc và phản hồi sự kiện**.

Việc tách **Integer PRF** và **Vector PRF** cũng cho thấy ARC không chỉ là một scalar core thuần điều khiển. Nó được trang bị để xử lý cả những đoạn mã có nhu cầu vector/FP đáng kể, nhưng không đánh đổi tài nguyên integer vốn rất quan trọng với mã điều phối. Cách bố trí này làm cho ARC trở thành một core đa dụng theo nghĩa đầy đủ: vừa đủ mạnh để làm host control plane, vừa đủ rộng để xử lý những đoạn tiền xử lý, hậu xử lý, hoặc thậm chí một phần compute không phù hợp để đẩy sang SPE.

Bảng **issue 24 μops** và **retirement 16 μops** tiếp tục cho thấy ARC được thiết kế để theo đuổi **thông lượng thực thi rất cao ở cấp một core đơn**. Với core host, điều này cực kỳ quan trọng. Nếu ARC chỉ giới điều phối logic nhưng phát ra công việc chậm, hoặc không xử lý nổi lượng trạng thái hệ thống đang thay đổi, thì toàn cụm SPE sẽ dễ bị ãi việc hoặc bị điều phối kém hiệu quả. Một bề rộng issue lớn kết hợp với cửa sổ OoO sâu giúp ARC duy trì áp lực ổn định lên back-end và giảm rủi ro nghẽn ở những đoạn mã có cấu trúc không đều.

4. Back-end integer: nền tảng thực sự của một core quản trị hệ thống tăng tốc

Khối integer của ARC Core rất đồ sộ. Sơ đồ cho thấy nhiều **INT Scheduler** tách cụm, gồm các cụm **32 entry** lặp lại nhiều lần, thêm các cụm **24 entry** và **48 entry**, đi kèm hàng loạt execution units kiểu **ALU**, **ALU Branch**, **ALU MUL**, **ALU MUL DIV**. Chỉ riêng cách tổ chức này cũng đã nói lên rằng ARC không phải loại core dựa quá nhiều vào “một vài cổng ALU chung chung”, mà là một thiết kế có **năng lực scalar rất dày**.

Đây là điều hoàn toàn hợp logic nếu ARC là core quản lý đám SPE. Phần mềm điều phối accelerator nhìn bề ngoài có vẻ không “nặng số học”, nhưng thực tế lại rất đòi hỏi ở integer side. Các công việc như **xây cấu trúc lệnh, thao tác con trỏ, duyệt queue, quản lý descriptor, tính toán offset, điều kiện hóa đường điều khiển, phân mảnh workload, xử lý trạng thái completion, kiểm tra tài nguyên, dịch lịch thực thi** đều dựa rất mạnh vào tài nguyên integer. Một core muốn làm tốt vai trò host phải có **branch throughput tốt, ALU bandwidth cao**, và đủ tài nguyên cho các phép toán nhân/chia khi cần xử lý địa chỉ hóa, kích thước khối, stride, chia việc hay các biểu thức runtime phức tạp.

Các cụm **ALU Branch** xuất hiện nhiều lần cho thấy thiết kế của ARC đặc biệt coi trọng phần rẽ nhánh và điều khiển. Đây là khác biệt bản chất so với một engine tăng tốc thuần data-plane như SPE, vốn thường hiệu quả nhất khi pipeline được nuôi bởi mã tương đối đều và ít nhánh. ARC thì ngược lại: nó phải “sống khỏe” trong thế giới của **if/else, loop, dispatch, exception path, completion path, wake-up logic**. Việc đầu tư dày vào tài nguyên integer vì thế là hợp lý và cần thiết.

Nói ngắn gọn, nếu SPE là nơi **ăn dữ liệu**, thì ARC là nơi **ra quyết định**. Và muốn ra quyết định nhanh ở quy mô hệ thống, integer side của ARC buộc phải rất mạnh. Sơ đồ cho thấy điều đó đã được phản ánh trực tiếp vào cấu trúc scheduler và execution units.

5. Khối FP/vector: ARC không chỉ là “bộ điều phối”, mà còn là một core lai rất rộng

Một điểm rất đáng chú ý ở ARC Core là dù vai trò trung tâm của nó thiên về điều phối, nó vẫn được trang bị **khối FP/vector rất dày**. Sơ đồ cho thấy có nhiều **52 FP Scheduler**, đi kèm các cụm thực thi kiểu **VALU / VMUL / FP ADD**, và thêm các khối **FP MUL / FP DIV** chuyên dụng. Điều này cho thấy ARC không hề bị giới hạn thành một scalar control core đơn thuần.

Ý nghĩa của việc này rất lớn. Trong một hệ thống có SPE, không phải lúc nào mọi tác vụ số học cũng nên hoặc cũng có thể đẩy sang cụm tăng tốc. Luôn tồn tại những phần việc như **tiền xử lý dữ liệu, hậu xử lý kết quả, vector math không đủ lớn để offload, các đoạn kernel lai giữa điều khiển và tính toán, reductions nhỏ, lọc dữ liệu theo điều kiện, chuyển đổi định dạng**, hoặc các đoạn công việc cần quyết định động trước khi biết có nên đẩy sang SPE hay không. ARC sở hữu tài nguyên FP/vector lớn sẽ giúp hệ thống tránh tình

trạng “việc gì hơi có số học cũng phải ném sang accelerator”, từ đó giảm overhead điều phối và tăng tính linh hoạt.

Ở góc độ thiết kế tổng thể, điều này cũng giúp ARC trở thành **một core cầu nối rất mạnh** giữa thế giới general-purpose và thế giới accelerator. Nó có đủ năng lực scalar để quản lý hệ thống, nhưng cũng có đủ tài nguyên vector để trực tiếp xử lý những phần việc mà nếu đẩy hẳn xuống SPE có thể không kinh tế. Đây là cách làm rất thông minh trong các kiến trúc dị thể: host core không nên chỉ là “quản lý giấy tờ”, mà phải đủ mạnh để tự gánh những đoạn mã trung gian và những phần không đáng để chuyển ngữ cảnh sang accelerator.

Tuy vậy, nhìn toàn cục thì khối FP/vector của ARC vẫn mang ý nghĩa **đa dụng, bao quát và hỗ trợ**, chứ không thay thế triết lý của SPE. SPE, theo sơ đồ của nó, là engine cực kỳ chuyên biệt cho local-store, DMA và pipeline vector dày đặc. ARC thì khác: nó có vector mạnh, nhưng nằm trong một cấu trúc OoO general-purpose rộng lớn hơn nhiều. Chính sự khác biệt này làm cho hai loại core **bổ sung cho nhau** thay vì chồng lấn hoàn toàn.

6. Memory subsystem: một trong những điểm then chốt khiến ARC phù hợp để quản lý SPE

Nếu phải chọn phần quan trọng nhất quyết định việc ARC có hợp làm host cho SPE hay không, thì đó rất có thể là **memory subsystem**. Sơ đồ cho thấy ARC sở hữu **128 MEM Scheduler**, hai **AGU Store**, hai **AGU Unified**, nhiều **AGU Load Data**, cùng **96 Store Buffer** và **160 Load Buffer**. Đây là một cấu hình cực kỳ mạnh về khả năng xử lý lưu lượng bộ nhớ, và nó phù hợp hoàn hảo với vai trò điều phối accelerator.

Quản lý một cụm SPE không chỉ là “ra lệnh”. Nó còn là xử lý một lượng lớn metadata và giao tiếp bộ nhớ: **đọc/ghi hàng đợi công việc, thao tác descriptor, chuẩn bị vùng dữ liệu, kiểm tra completion, quản lý trạng thái đồng bộ, phối hợp DMA, cập nhật con trỏ vòng, đưng chạm nhiều cấu trúc dữ liệu dùng chung**. Những workload kiểu đó không phải lúc nào cũng có locality lý tưởng, và thường đòi hỏi giữ nhiều memory operation đang bay đồng thời. Một **MEM Scheduler 128 entry** cộng với **load/store buffers rất lớn** cho phép ARC theo dõi và hấp thụ lượng lớn thao tác nhớ mà không làm pipeline dễ dàng nghẽn cứng.

Sự hiện diện của nhiều **AGU** cũng rất quan trọng. Nó cho thấy ARC có thể phát sinh và xử lý song song nhiều luồng địa chỉ cho load/store, điều đặc biệt cần khi host phải cùng lúc xử

lý cấu trúc điều phối, vùng dữ liệu chia sẻ, và các luồng trạng thái liên quan tới accelerator. Nếu thiếu năng lực AGU, một core dù mạnh ở ALU vẫn dễ bị bóp nghẹt khi bước vào vùng code quản trị bộ nhớ dày đặc.

Tóm lại, ARC Core không chỉ được dựng để “tính nhanh”, mà còn được dựng để **di chuyển, theo dõi và tổ chức dữ liệu nhanh**. Và đó chính là đặc tính sống còn của một host core cho cụm SPE.

7. Cache hierarchy và TLB: nền móng cho tính ổn định ở cấp hệ thống

Hệ thống cache và TLB của ARC rất giàu tài nguyên, đúng với vị thế của một core trung tâm. Ở tầng gần nhất, sơ đồ cho thấy **L1 DTLB 384 entry** và **128KB L1 D-Cache (8-way)**. Một **L1D 128KB** là cực lớn đối với một core đơn, cho thấy nhà thiết kế muốn giảm tối đa áp lực miss ở tầng gần nhất. Điều này rất có ích khi ARC phải thường xuyên đụng tới **queue, descriptor, bảng trạng thái, cấu trúc metadata, bảng ánh xạ, bộ đệm điều phối** và nhiều dữ liệu “nóng” nhưng rải rác.

Bên trên L1 là một tầng **pL2 Cache 1MB** cùng **pL2 TLB 640 entry**, rồi tiếp tục lên **L2 TLB 4096 entry** và **16MB Shared L2 Cache (16-way)**. Cấu trúc này cho thấy ARC không chỉ mạnh ở cache gần, mà còn có một **hệ phân cấp bộ nhớ sâu và rộng**, rất phù hợp để đóng vai trò trung tâm trong một hệ thống có accelerator. Một core host quản lý SPE thường phải chạm vào nhiều lớp dữ liệu với quy mô khác nhau: dữ liệu cực nóng ở L1, metadata và command structures ở tầng gần, rồi các working set lớn hơn ở shared L2. Việc có **private mid-level cache** trước khi ra **shared L2 lớn** giúp ARC giữ lại những gì đặc thù của riêng nó, đồng thời vẫn có nơi chia sẻ dung lượng lớn hơn cho toàn hệ thống.

L2 TLB 4096 entry đặc biệt đáng giá. Nó cho thấy ARC được chuẩn bị để làm việc với **không gian địa chỉ lớn, nhiều mapping, nhiều vùng nhớ hoạt động đồng thời**. Với một core điều phối accelerator, đây là điều gần như bắt buộc: càng nhiều luồng việc, càng nhiều vùng dữ liệu, càng nhiều đối tượng đang hoạt động, áp lực lên TLB càng tăng. Nếu TLB không đủ sâu, mọi sức mạnh front-end và execution phía trên đều bị phí phạm.

Một điểm nữa rất đáng chú ý là các đường **32 B/cycle** và **64 B/cycle** trong khối bộ nhớ. Chúng cho thấy dung lượng không phải là thứ duy nhất được đầu tư; **băng thông giữa các tầng** cũng được cân nhắc cẩn thận. Cache lớn mà dữ liệu không luân chuyển đủ nhanh thì

vẫn không phục vụ tốt một core host cấp cao. Sơ đồ của ARC cho thấy nhà thiết kế hiểu rõ điều đó.

8. Vì sao ARC Core rất hợp để quản lý một cụm SPE

Khi đặt sơ đồ ARC cạnh sơ đồ SPE, mối quan hệ giữa hai khối trở nên khá tự nhiên. **SPE** có cấu trúc nghiêng mạnh về **pipeline vector, load/store chuyên dụng, local store 2MB và DMA**, tức là nó rất phù hợp để trở thành **data-plane engine**: nhận công việc đã được chuẩn bị, kéo dữ liệu vào local store, xử lý song song rất mạnh, rồi trả kết quả ra. Nhưng chính vì chuyên biệt như vậy, SPE không phải nơi lý tưởng cho những phần việc “lộn xộn” của hệ thống.

Đó là lý do **ARC Core** trở thành đối tác hoàn hảo. Một host core quản lý SPE cần làm được ít nhất bốn việc thật tốt.

Thứ nhất, nó phải **xử lý mã điều khiển cực tốt**, vì điều phối accelerator luôn kéo theo nhiều logic động. Predictor và front-end của ARC cho thấy nó rất mạnh ở việc này.

Thứ hai, nó phải **chịu được độ trễ và sự không đều của bộ nhớ**, vì quản lý queue, descriptor, completion và các cấu trúc chia sẻ không bao giờ “mượt” như một kernel dense linear algebra. Cửa sổ OoO sâu, ROB lớn, load/store buffers dày và cache/TLB rộng giúp ARC làm đúng vai đó.

Thứ ba, nó phải có **đủ năng lực scalar lẫn vector** để tự gánh những phần việc không đáng đẩy sang SPE. ARC làm được điều đó nhờ back-end integer dày cùng cụm FP/vector rất mạnh.

Thứ tư, nó phải có **quy mô tài nguyên đủ lớn để nuôi cả một cụm accelerator mà không tự trở thành nút thắt cổ chai**. Bề rộng decode, issue, memory scheduling và cache hierarchy của ARC cho thấy nó không phải “một core host tượng trưng”, mà là một core đủ lớn để trở thành trung tâm điều phối thật sự.

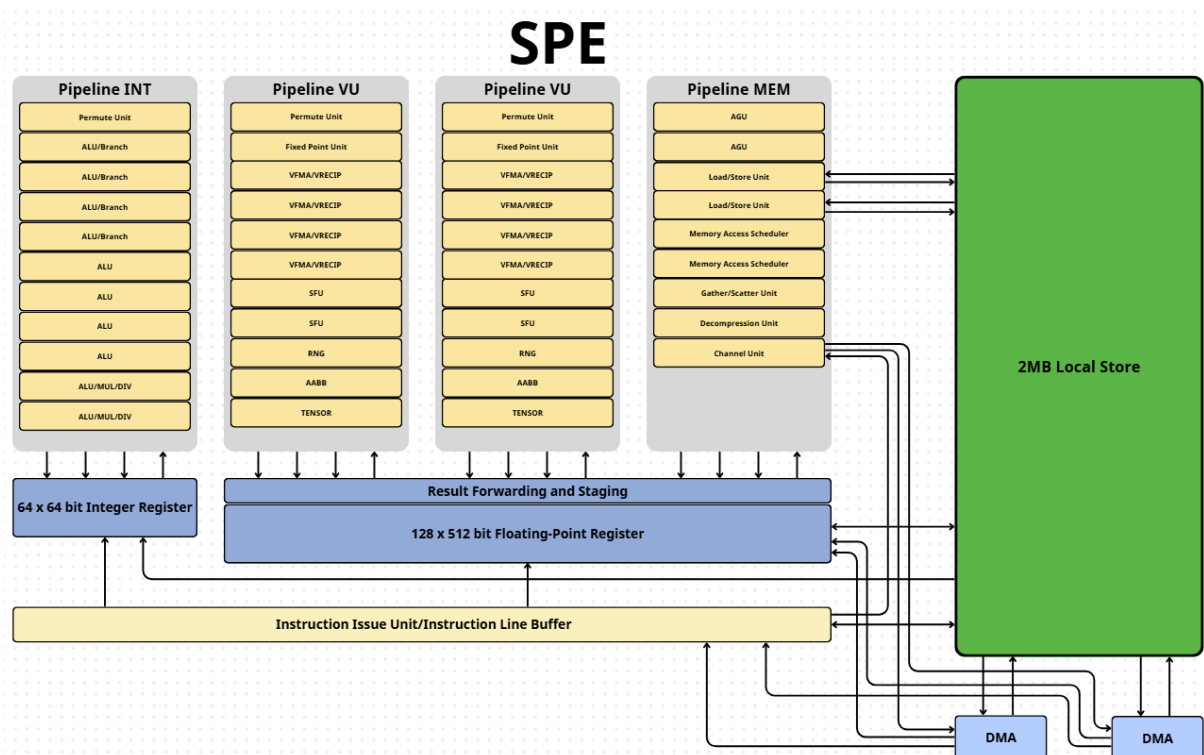
Theo cách hiểu đó, ARC không chỉ “quản lý” SPE theo nghĩa hành chính. Nó là lớp xử lý quyết định xem **cái gì nên chạy ở đâu, khi nào, với dữ liệu nào, theo thứ tự nào, và phải đồng bộ ra sao**. Đây là một vai trò kiến trúc rất nặng, và sơ đồ ARC cho thấy nó đã được thiết kế đúng theo yêu cầu đó.

9. Kết luận

Nếu viết thật cô đọng nhưng vẫn đúng tinh thần của toàn bộ sơ đồ, thì **ARC Core** là một **core general-purpose** cực rộng, cực sâu và cực giàu tài nguyên, được thiết kế để đóng vai trò **bộ não điều phối trung tâm** trong một hệ thống có cụm **SPE** làm engine tăng tốc. Nó có **front-end rất mạnh**, **branch prediction đẳng cấp cao**, **trace cache**, **decode rộng**, **ROB rất sâu**, tài nguyên integer và FP/vector dày, cùng **memory subsystem** và **cache hierarchy thuộc loại rất lớn**. Tất cả những điều đó kết hợp lại thành một kết luận khá rõ ràng: ARC không phải được dựng lên chỉ để “thêm một core mạnh”, mà để trở thành **lớp control-plane thực thụ** cho toàn bộ cụm tính toán chuyên biệt phía sau.

Giá trị lớn nhất của ARC vì thế không nằm ở một con số đơn lẻ như ROB hay cache, mà nằm ở **sự cân bằng rất có chủ đích giữa điều khiển, thực thi và bộ nhớ**. Nó đủ giỏi ở điều khiển để quản lý SPE, đủ mạnh ở thực thi để không bị các đoạn mã hỗn hợp làm chậm, và đủ rộng ở bộ nhớ để hấp thụ áp lực điều phối dữ liệu của cả hệ thống. Nói cách khác, nếu SPE là cánh tay chuyên dụng để xử lý dữ liệu thật nhanh, thì **ARC Core chính là bộ não quyết định cách toàn bộ cánh tay đó vận hành**.

SPE



1. Tổng thể kiến trúc và vai trò của SPE

Nếu **ARC Core** là bộ não điều phối, thì **SPE** trong sơ đồ này rõ ràng là phần **động cơ tăng tốc chuyên dụng** của toàn hệ thống. Cách tổ chức của SPE khác rất xa một CPU core kiểu truyền thống: nó không đặt trọng tâm vào branch prediction phức tạp, cache hierarchy nhiều tầng hay một cửa sổ out-of-order khổng lồ. Thay vào đó, toàn bộ cấu trúc được dựng lên để phục vụ một mục tiêu rất rõ: **biến dữ liệu thành kết quả với mật độ tính toán cực cao, độ đều cao, và overhead điều khiển thấp nhất có thể**.

Nhìn vào sơ đồ, SPE mang bản chất của một **compute engine thiên về data-plane** hơn là một nhân general-purpose. Nó có các pipeline được phân vai rất rõ, có cụm vector cực dày, có khối memory riêng chuyên thao tác dữ liệu, có **local store 2MB** thay vì phụ thuộc hoàn toàn vào cache truyền thống, và có **DMA** để chủ động di chuyển dữ liệu. Tức là SPE không chờ hệ thống “tự nuôi” nó như một CPU core thông thường; nó được xây để **nhận việc, kéo dữ liệu vào không gian cục bộ, xử lý song song rất mạnh, rồi đẩy kết quả ra**.

Đó cũng là lý do SPE đặc biệt phù hợp với những workload như **vector math, tensor-style compute, xử lý tín hiệu, media, graphics-adjacent compute, AI kernels, physics, nén/giải nén, transform dữ liệu, và các tác vụ streaming có cấu trúc rõ ràng**. Những loại việc này thường không cần khả năng xử lý nhánh quá tinh vi, nhưng lại cần lượng toán tử dày, băng dữ liệu lớn, và khả năng giữ pipeline luôn bận. SPE được dựng chính xác cho kiểu bài toán đó.

2. Triết lý thiết kế: từ “CPU xử lý mọi thứ” sang “engine làm thật nhanh một lớp việc cụ thể”

Điểm quan trọng nhất khi đọc SPE là phải nhìn nó như một **khối tăng tốc chuyên biệt**, chứ không phải một “CPU core nhỏ đi”. Một CPU core lớn như ARC được tối ưu để chịu được mã điều khiển phức tạp, đường đi lệnh không đều, nhiều nhánh, nhiều trạng thái, và bộ nhớ khó đoán. SPE thì đi theo hướng ngược lại: nó giả định rằng phần việc đã được chuẩn bị tương đối tốt từ phía host, dữ liệu đã được phân khối hợp lý, và giờ mục tiêu là **đẩy throughput của kernel tính toán lên càng cao càng tốt**.

Chính vì thế, sơ đồ SPE không xoay quanh predictor, ROB hay scheduler đa tầng kiểu CPU lớn. Thay vào đó, nó xoay quanh các thành phần như **Pipeline INT, hai Pipeline VU**,

Pipeline MEM, register files lớn, result forwarding/staging, instruction issue/line buffer, và đặc biệt là **2MB Local Store**. Kiểu tổ chức này cho thấy nhà thiết kế muốn tách bạch rất rõ các dòng công việc: integer/control nội bộ, vector compute, và data movement.

Về bản chất, SPE được tối ưu cho mô hình **explicit data orchestration**. ARC Core sẽ quyết định công việc nào giao cho SPE, cấu trúc dữ liệu ra sao, thời điểm nào chuyển dữ liệu, và kernel nào sẽ chạy. Còn SPE sẽ tập trung làm phần việc mà nó giỏi nhất: **xử lý một lượng lớn dữ liệu theo cấu trúc lặp, đều, và song song hóa mạnh**. Đây là một mô hình rất hiệu quả khi hệ thống muốn đạt hiệu năng cao mà vẫn giữ được sự phân vai rõ ràng giữa “não điều phối” và “cỗ máy tính toán”.

3. Front-end của SPE: không rộng theo kiểu CPU, mà gọn để nuôi pipeline chuyên dụng

SPE không có một front-end kiểu CPU out-of-order nặng về dự đoán nhánh và decode siêu phức tạp. Thay vào đó, sơ đồ cho thấy nó có một khối **Instruction Issue Unit / Instruction Line Buffer** nằm ở trung tâm phía dưới, đóng vai trò như bộ phát lệnh cho toàn bộ các pipeline phía trên. Điều này nói rất nhiều về cách vận hành của SPE.

Thay vì cố xử lý mã có điều khiển phức tạp và cấu trúc động như ARC Core, SPE nhiều khả năng được tối ưu cho các **instruction streams đều đặn, lặp lại, có locality tốt ở cấp kernel**. “Instruction Line Buffer” gợi ý rằng phần lệnh được tổ chức và cấp phát theo cách tối ưu cho những đoạn kernel nóng, lặp nhiều, với overhead fetch/issue thấp. Đây là cách làm rất hợp lý cho accelerator: khi một kernel đã được đẩy vào SPE, phần quan trọng không còn là “đoán nhánh cho giỏi”, mà là **duy trì tốc độ cấp lệnh đều đặn cho các pipeline tính toán và memory**.

Cách tổ chức như vậy cũng giúp giảm đáng kể độ phức tạp điều khiển nội bộ. Một CPU core lớn phải dành rất nhiều transistor cho predictor, rename, rollback, recovery và tracking vi lệnh. SPE thì chuyển phần phức tạp đó lên phía ARC, còn bản thân nó giữ front-end ở dạng gọn hơn, trực tiếp hơn, tối ưu cho **kernel execution**. Đó là lý do accelerator thường có hiệu suất trên diện tích và trên điện năng rất cao khi workload phù hợp: nó không phải “gánh” toàn bộ chi phí của một CPU tổng dụng.

4. Pipeline INT: lớp điều khiển và integer support cho kernel

Khối **Pipeline INT** của SPE cho thấy dù đây là một engine tính toán chuyên biệt, nó vẫn không bỏ qua phần integer/control. Trong sơ đồ, pipeline này có **Permute Unit**, nhiều khối **ALU/Branch**, nhiều **ALU**, cùng các đơn vị **ALU/MUL/DIV**. Cấu hình đó cho thấy SPE vẫn có đủ khả năng xử lý logic điều khiển cục bộ, số học nguyên, thao tác địa chỉ, và các biến đổi dữ liệu kiểu integer ngay trong lòng accelerator.

Điều này rất quan trọng. Một engine vector mạnh không thể vận hành hiệu quả nếu mọi phần việc điều khiển nhỏ đều phải quay lại host. Trong thực tế, ngay cả các kernel đậm đặc vector cũng cần nhiều thao tác như **index calculation, loop bookkeeping, address arithmetic, mask/control decisions, integer transforms, branch cục bộ, permutation và chuẩn bị toán hạng**. Pipeline INT của SPE chính là lớp hạ tầng giúp kernel tự đứng vững mà không liên tục phụ thuộc vào ARC cho từng chi tiết nhỏ.

Sự hiện diện của **Permute Unit** ở pipeline INT cũng rất đáng chú ý. Nó cho thấy SPE được tối ưu không chỉ cho “tính toán”, mà còn cho **tổ chức lại dữ liệu**. Trong rất nhiều workload vector, việc đổi trật tự phần tử, ghép/tách lane, align dữ liệu, hay tái bố trí khối dữ liệu có vai trò cực lớn. Một permute unit mạnh thường là dấu hiệu của một kiến trúc nhắm tới các bài toán streaming, SIMD, signal processing hoặc tensor/data transforms, nơi **data layout** quan trọng không kém bản thân phép toán.

Nói cách khác, Pipeline INT trong SPE không phải phần phụ mờ nhạt. Nó là lớp điều khiển nội bộ giúp accelerator đủ linh hoạt để chạy kernel thực tế, chứ không chỉ những biểu thức vector lý tưởng trên giấy.

5. Hai Pipeline VU: trung tâm sức mạnh tính toán của SPE

Trọng tâm thực sự của SPE nằm ở **hai Pipeline VU**. Chỉ cần nhìn vào mật độ đơn vị chức năng trong hai cột này là có thể thấy SPE được xây để đẩy thông lượng vector lên rất cao. Mỗi pipeline VU đều có **Permute Unit, Fixed Point Unit**, nhiều khối **VFMA/VRECIP**, các **SFU**, rồi thêm các khối đặc biệt như **RNG, AABB và TENSOR**. Đây là một tổ hợp cực kỳ giàu ý nghĩa.

Đầu tiên, sự lặp lại của các khối **VFMA/VRECIP** cho thấy SPE được tối ưu mạnh cho các phép toán kiểu **vector fused multiply-add**, nghịch đảo gần đúng, và các kernel số học có

tính lập cao. FMA luôn là dấu hiệu của một engine nhắm tới **throughput compute đậm đặc**, vì rất nhiều workload hiện đại từ đồ họa, mô phỏng vật lý, xử lý tín hiệu đến machine learning đều dựa rất mạnh vào multiply-accumulate. Việc có nhiều đơn vị loại này trên mỗi VU pipeline cho thấy SPE không chỉ “có vector”, mà là **vector ở mức chủ lực**.

Fixed Point Unit cho thấy accelerator này không chỉ phục vụ FP thuần túy. Nó còn có không gian cho các workload dùng toán fixed-point, vốn rất quan trọng trong media, DSP, inference định lượng thấp, xử lý ảnh, mã hóa/giải mã, và nhiều pipeline data processing cần hiệu suất năng lượng cao hơn floating-point đầy đủ. Điều này làm SPE trở nên linh hoạt hơn rất nhiều so với một engine chỉ mạnh ở FP32/FP64 truyền thống.

Các **SFU** cho thấy SPE có hỗ trợ cho những phép toán đặc biệt thường gặp trong đồ họa, scientific compute hoặc AI, nơi các hàm toán học ngoài nhóm cộng/nhân cơ bản có thể xuất hiện với tần suất đủ cao để xứng đáng có phần cứng riêng. Điều này giúp giảm độ trễ và tăng throughput cho những kernel chuyên biệt.

Khối **RNG** là một chi tiết rất thú vị. Nó gợi ý rằng SPE có thể phục vụ tốt các workload cần sinh số ngẫu nhiên hoặc pseudo-random tại chỗ, chẳng hạn một số dạng mô phỏng, Monte Carlo, sampling, procedural generation hoặc AI/data augmentation. Tương tự, **AABB** cho thấy kiến trúc có thể đã tính tới các bài toán hình học, va chạm, spatial processing hoặc một số dạng acceleration liên quan tới bounding volumes. Còn khối **TENSOR** là tín hiệu rất rõ rằng SPE không chỉ là vector engine cổ điển, mà có định hướng rõ rệt tới **AI-style compute** hoặc các phép toán tensor hóa hiện đại.

Tổng hợp lại, hai Pipeline VU cho thấy SPE là một **compute fabric cực kỳ dày**, không bó hẹp vào một miền nhỏ. Nó có thể phục vụ từ vector math truyền thống cho tới AI/tensor-centric kernels, từ fixed-point tới floating-point, từ data permutation tới special functions. Đây chính là trái tim hiệu năng của toàn bộ SPE.

6. Register files: thiết kế để nuôi song song hóa ở mật độ cao

Một accelerator chỉ mạnh khi nó có đủ không gian lưu trạng thái và toán hạng để giữ các pipeline luôn bận. Sơ đồ SPE cho thấy điều đó được đầu tư rất rõ qua hai khối thanh ghi lớn: **64 × 64-bit Integer Register** và **128 × 512-bit Floating-Point Register**.

Cụm **64 thanh ghi integer 64-bit** là nền tảng cho mọi phần logic điều khiển cục bộ, địa chỉ hóa, loop counters, mask handling và integer transforms. Nó cho phép kernel giữ nhiều trạng thái hoạt động ngay trong register file mà không phải thường xuyên quay ra bộ nhớ cục bộ. Với một accelerator, đây là điểm rất quan trọng vì mỗi lần phải rời register để truy cập bộ nhớ đều làm tăng áp lực lên data path.

Khối **128 thanh ghi dấu chấm động 512-bit** mới thực sự cho thấy SPE nghiêng mạnh tới vector throughput. Một register file như vậy cho phép giữ lượng lớn vector operands, partial sums, tiles dữ liệu, intermediate values và mask/state liên quan tới SIMD ngay bên trong lõi tính toán. 512-bit là độ rộng rất lớn, và khi kết hợp với hai pipeline VU dày đặc, nó cho thấy SPE được thiết kế để khai thác **vector width lớn lẫn vector depth lớn** cùng lúc.

Điểm đáng nói là register file lớn không chỉ giúp tăng raw throughput; nó còn giúp giảm nhu cầu truy cập local store cho các giá trị trung gian. Trong các kernel tối ưu tốt, dữ liệu có thể được kéo vào, biến đổi qua nhiều bước trong register, rồi mới ghi ra. Khi đó, hiệu suất của accelerator tăng lên rất mạnh vì local store và DMA được dùng chủ yếu cho dữ liệu đầu vào/đầu ra, thay vì bị tiêu hao vào việc đẩy qua lại trạng thái trung gian.

7. Result Forwarding and Staging: chìa khóa để giữ pipeline không bị “đói”

Khối **Result Forwarding and Staging** nằm giữa các pipeline thực thi và register file là một thành phần cực kỳ quan trọng, dù nhìn qua có thể không “hào nhoáng” bằng các đơn vị tensor hay VFMA. Về bản chất, đây là lớp hạ tầng để SPE **luân chuyển kết quả nhanh giữa các pipeline**, giảm stall, giảm latency nhìn thấy bởi các lệnh phụ thuộc sau, và duy trì mật độ sử dụng execution units ở mức cao.

Trong một accelerator nhiều pipeline, bài toán không chỉ là “có bao nhiêu đơn vị tính”, mà còn là **kết quả có được chuyển đủ nhanh đến nơi cần dùng tiếp theo hay không**. Nếu không có forwarding/staging tốt, các pipeline mạnh đến đâu cũng dễ bị nghẽn do phải chờ ghi-back rồi đọc lại. Khối này cho thấy SPE được thiết kế với tư duy rất thực tế: hiệu năng bền vững đến từ **đường dữ liệu nội bộ sạch, ngắn và ít ma sát**, chứ không chỉ từ số lượng unit.

Đặc biệt với các kernel vector, tensor hoặc chuỗi phép toán có phụ thuộc dày, forwarding tốt giúp giữ nhịp pipeline ổn định hơn rất nhiều. Điều này rất phù hợp với toàn bộ triết lý của

SPE: một khi kernel đã vào máy, hệ thống phải cố đẩy nó chạy ở mức đều và sát đỉnh nhất có thể.

8. Pipeline MEM: dữ liệu là nhiên liệu, và SPE được xây để chủ động quản lý nhiên liệu đó

Một trong những phần quan trọng nhất của SPE là **Pipeline MEM**. Sơ đồ cho thấy khối này gồm **2 AGU**, **2 Load/Store Unit**, **2 Memory Access Scheduler**, thêm **Gather/Scatter Unit**, **Decompression Unit**, và **Channel Unit**. Đây là một tập hợp rất giàu ngụ ý, vì nó cho thấy SPE không chỉ là một khối compute mạnh, mà còn là một engine **data movement** được đầu tư nghiêm túc.

Hai **AGU** và hai **Load/Store Unit** cho thấy accelerator có thể xử lý song song việc tạo địa chỉ và luân chuyển dữ liệu giữa local store với các pipeline tính toán. Điều này rất quan trọng trong các kernel streaming, nơi hiệu năng thường bị giới hạn bởi khả năng “nuôi” compute units bằng dữ liệu liên tục.

Sự xuất hiện của **Memory Access Scheduler** theo cặp cho thấy luồng truy cập bộ nhớ không được xử lý một cách đơn giản, mà có tổ chức và điều phối riêng. Điều đó hợp lý với các accelerator có độ song song cao: nếu không có lớp điều phối truy cập bộ nhớ đủ thông minh, bản thân compute fabric rất mạnh của SPE sẽ nhanh chóng bị thiếu dữ liệu.

Gather/Scatter Unit là một điểm cực kỳ đáng giá. Nó cho thấy SPE không chỉ mạnh khi dữ liệu nằm thẳng tắp, contiguous và hoàn hảo. Nó còn được chuẩn bị cho các trường hợp truy cập **không đều, gián đoạn, hoặc cần gom/tán phần tử theo mẫu**. Đây là khả năng quan trọng trong AI, đồ họa, hình học, sparse-ish workloads, và rất nhiều pipeline dữ liệu hiện đại vốn không còn đơn giản như đọc một dải liên tục rồi nhân cộng. **Decompression Unit** cho thấy kiến trúc đã tính tới khả năng dữ liệu có thể được nén khi di chuyển hoặc lưu trữ, rồi được giải nén gần nơi tính toán. Nếu đúng như sơ đồ gợi ý, đây là một điểm rất mạnh: nó giúp giảm áp lực băng thông ở ngoài, đổi lại giải nén cục bộ ngay trong accelerator. Điều này đặc biệt có ý nghĩa với các hệ thống mà compute rẻ hơn băng thông.

Cuối cùng, **Channel Unit** nhiều khả năng đóng vai trò cầu nối dữ liệu và điều khiển giữa local store, DMA và các thành phần bên ngoài. Nó khiến SPE giống một thực thể xử lý khá độc lập, có khả năng giao tiếp và vận hành dữ liệu theo cơ chế riêng của mình, chứ không chỉ là “một vài ALU gắn thêm bên cạnh CPU”.

9. 2MB Local Store: trái tim bản sắc của SPE

Nếu phải chọn một chi tiết định nghĩa SPE mạnh nhất, thì đó chính là **2MB Local Store**. Khối này không chỉ lớn về mặt dung lượng; nó còn nói lên cả triết lý kiến trúc. SPE không sống chủ yếu bằng cache trong suốt như CPU core thông thường. Nó sống bằng một vùng nhớ cục bộ lớn, gần, và được quản lý có chủ đích hơn.

Ý nghĩa của **local store** là cực lớn. Nó cho phép dữ liệu được đưa sát tới compute engine, giữ ở đó dưới sự kiểm soát chặt chẽ của phần mềm/hardware runtime, rồi được tái sử dụng với độ trễ thấp và băng thông nội bộ cao. Với những kernel có cấu trúc rõ, đây là cách tiếp cận rất mạnh vì nó giảm sự bất định vốn thường đi kèm cache tự động. Thay vì chờ xem dữ liệu có “may mắn” nằm trong cache hay không, hệ thống có thể **chủ động staging dữ liệu** vào local store trước khi tính toán.

Điều này đặc biệt phù hợp với mô hình mà **ARC Core quản lý SPE**. ARC có thể chia việc thành các tile, chuẩn bị descriptor, điều khiển DMA kéo từng khối dữ liệu vào **2MB Local Store**, rồi SPE xử lý tại chỗ bằng các pipeline VU/INT/MEM của nó. Sau đó kết quả được ghi ra hoặc chuyển tiếp cho chặng tiếp theo. Đây là mô hình kinh điển của những engine tăng tốc rất mạnh: **đưa dữ liệu đến gần nơi tính toán, rồi tận dụng tối đa locality nhân tạo do phần mềm/runtime tạo ra**.

Về mặt hiệu năng, local store thường là chìa khóa để đạt **throughput rất cao và rất ổn định**, miễn là phần mềm điều phối đủ tốt. Đó cũng là lý do ARC và SPE phải đi cùng nhau: ARC lo phần việc phức tạp của điều phối, còn SPE hưởng lợi từ dữ liệu đã được sắp xếp và cấp phát đúng lúc.

10. DMA: SPE không thụ động chờ dữ liệu, nó chủ động kéo dữ liệu

Sơ đồ cho thấy SPE có **hai khối DMA** kết nối với local store. Đây là chi tiết có ý nghĩa nền tảng. Một accelerator muốn thật sự mạnh không thể hoàn toàn phụ thuộc vào load/store kiểu CPU để lấy dữ liệu từ bộ nhớ xa. Nó cần cơ chế **bulk data movement** hiệu quả, tách khỏi đường thực thi thông thường. DMA chính là câu trả lời.

Với DMA, SPE có thể kéo các khối dữ liệu lớn vào local store và đẩy kết quả ra ngoài với overhead thấp hơn nhiều so với việc dùng từng lệnh load/store nhỏ lẻ. Điều này không chỉ

tiết kiệm năng lượng mà còn mở ra khả năng **chồng lấp giữa truyền dữ liệu và tính toán**. Trong một pipeline tối ưu tốt, khi một tile dữ liệu đang được xử lý trong compute engine, tile kế tiếp có thể được DMA kéo vào, và tile trước đó có thể đang được DMA đẩy kết quả ra. Kiểu vận hành này chính là con đường dẫn tới hiệu suất thực tế rất cao.

Việc có **hai DMA** còn gợi ý khả năng tăng song song hóa ở data movement, ví dụ tách luồng vào/ra, hoặc phục vụ nhiều hàng dữ liệu/luồng tác vụ khác nhau. Dù chi tiết chính xác của chính sách điều phối không hiện hết trên sơ đồ, chỉ riêng việc có nhiều DMA cũng cho thấy dữ liệu được xem là yếu tố sống còn của SPE, chứ không phải chuyện phụ.

Nói một cách hình ảnh, nếu compute units là các nhà máy thì **DMA là hệ thống vận tải công nghiệp**. Không có nó, local store sẽ không được nuôi đều, và toàn bộ sức mạnh của các VU pipeline sẽ bị lãng phí.

11. SPE mạnh ở đâu, và vì sao nó cần ARC đi cùng

Từ toàn bộ cấu trúc, có thể thấy SPE sẽ cực mạnh ở những bài toán có các đặc trưng sau: **dữ liệu nhiều, cấu trúc lặp rõ, song song hóa tốt, ít phụ thuộc vào nhánh phức tạp, và có thể chia thành block/tile để staging vào local store**. Các workload như vector/tensor compute, AI kernels, xử lý tín hiệu, media transforms, physics kernels, toán ma trận, filtering, compression/decompression pipelines, hoặc các phép xử lý hình học là những ứng viên rất phù hợp.

Ngược lại, SPE sẽ không phải lựa chọn tối ưu cho các phần việc đậm đặc điều khiển như runtime scheduling, quản lý queue phức tạp, xử lý nhiều ngoại lệ, hoặc các đoạn mã có nhánh rất khó đoán và working set trạng thái “lộn xộn”. Những phần đó nên được ARC xử lý. Và đó chính là logic chia vai của toàn kiến trúc: **ARC xử lý phần “nghĩ”, SPE xử lý phần “làm”**.

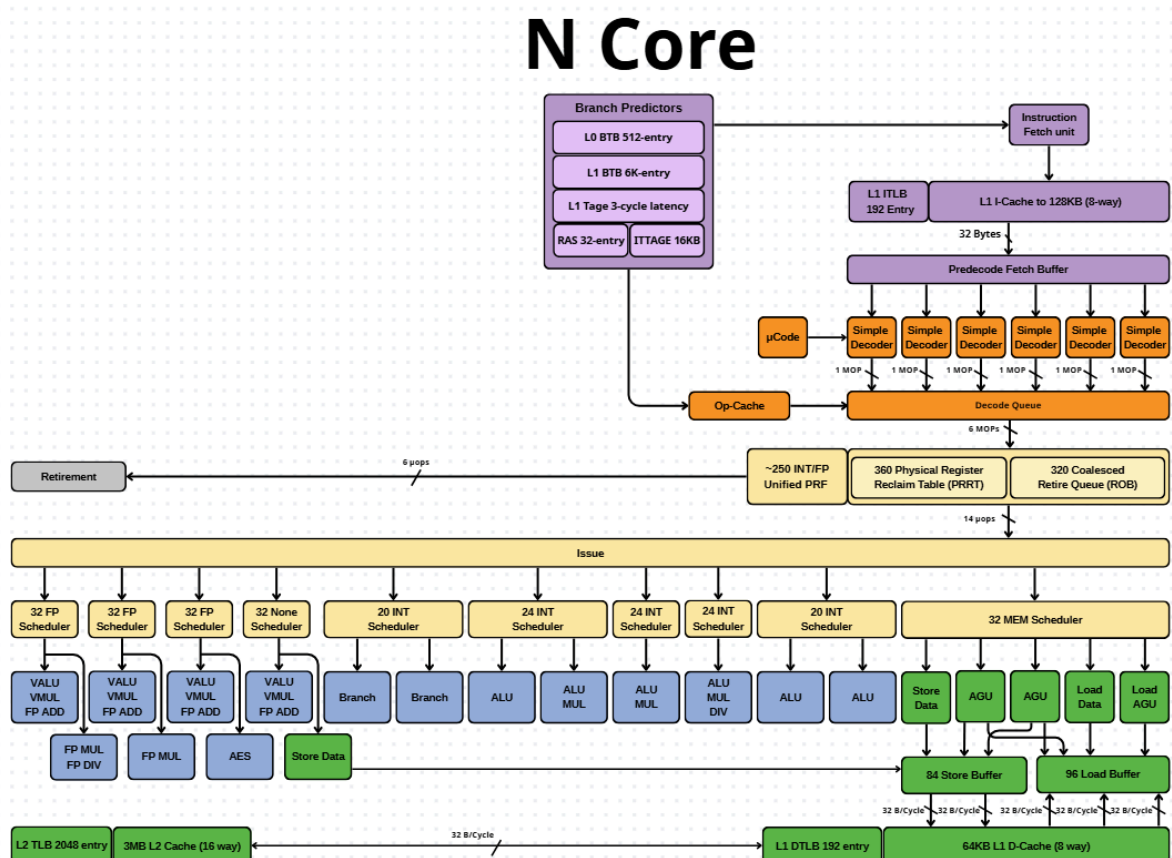
Một hệ thống chỉ có ARC sẽ rất linh hoạt nhưng kém hiệu quả hơn ở các kernel data-parallel dày đặc. Ngược lại, một hệ thống chỉ có SPE sẽ rất mạnh ở tính toán thuần nhưng gặp khó khi phải điều phối workload thực tế. Ghép hai thứ lại, ta có một kiến trúc mà mỗi bên làm điều mình giỏi nhất: ARC chuẩn bị, chia việc, đồng bộ và điều khiển; SPE hấp thụ dữ liệu và đẩy throughput tính toán lên mức tối đa.

12. Kết luận

Nếu tóm gọn nhưng vẫn giữ đúng tinh thần của sơ đồ, thì **SPE** là một **engine tăng tốc chuyên biệt, thiên về vector/tensor compute và data movement có chủ đích**, được thiết kế để nhận việc từ **ARC Core**, xử lý dữ liệu với mật độ cực cao trong **2MB Local Store**, rồi trả kết quả ra ngoài bằng cơ chế **DMA** và pipeline memory riêng. Nó không cố trở thành một CPU general-purpose; ngược lại, toàn bộ sức mạnh của nó đến từ việc **hy sinh tính tổng dụng để đổi lấy throughput, hiệu suất năng lượng và mật độ tính toán vượt trội** trên đúng lớp workload mà nó nhắm tới.

Điểm mạnh nhất của SPE nằm ở **hai Pipeline VU rất dày, register files lớn, Pipeline MEM giàu tính năng, gather/scatter, decompression, tensor-oriented units**, cùng **local store + DMA** như một nền tảng dữ liệu riêng. Đây là một cấu trúc rất “sạch” về mặt vai trò: ARC quản trị và quyết định, còn SPE biến dữ liệu thành kết quả với tốc độ cực cao.

N core



N Core trong sơ đồ này có thể được mô tả như một lõi CPU **ARM server-class** theo triết lý **general-purpose high-performance core**: một lõi tổng dụng, ngoài thứ tự, độ rộng lớn,

được tối ưu để gánh trực tiếp các workload trung tâm của máy chủ thay vì chỉ đóng vai trò điều phối hay offload tác vụ cho các đơn vị xử lý chuyên biệt. Nếu nhìn tổng thể từ frontend, rename/retire, backend thực thi cho tới hệ thống nhớ, đây là một kiến trúc được cân chỉnh rất rõ theo các ưu tiên của CPU máy chủ hiện đại: **khả năng duy trì throughput cao trên mã có nhánh rẽ phức tạp, che giấu độ trễ bộ nhớ, giữ nhiều lệnh “in-flight”, và đáp ứng tốt đồng thời các workload integer, vector/floating-point, và mã hóa.**

Điểm đầu tiên gây ấn tượng là **frontend** của N Core được xây theo hướng rất mạnh tay, cho thấy nhà thiết kế không xem việc tìm và nạp lệnh chỉ là một bước đầu vào đơn giản, mà coi đây là mặt trận quyết định hiệu năng thực tế. Cụm **branch predictor** gồm **L0 BTB 512-entry, L1 BTB 6K-entry, L1 TAGE 3-cycle latency, RAS 32-entry** và **ITTAGE 16KB** cho thấy lõi này được tối ưu cho các luồng lệnh có hành vi rẽ nhánh dày đặc, nhiều lời gọi hàm, nhiều nhánh gián tiếp và các mẫu thực thi khó dự đoán. Trong môi trường **ARM server**, điều này đặc biệt quan trọng, bởi phần lớn phần mềm máy chủ thực tế — từ kernel, hypervisor, container runtime, web server, database engine cho tới middleware và framework ứng dụng — đều có control flow phức tạp hơn rất nhiều so với các benchmark tính toán thuần túy. Một predictor mạnh không chỉ cải thiện IPC trung bình, mà còn giúp giảm tail latency, giữ hệ thống ổn định hơn khi tải tăng cao và khi working set mã bắt đầu trở nên phân tán.

Tầng nạp lệnh tiếp theo cũng phản ánh rất rõ định hướng đó. N Core có **Instruction Fetch Unit, L1 ITLB 192 entry** và **L1 I-Cache 128KB, 8-way**, sau đó là **Predecode Fetch Buffer** trước khi vào cụm giải mã. Một **I-cache 128KB** là khá rộng cho một core tổng dụng, và điều này hàm ý rằng kiến trúc nhắm đến khả năng giữ lại lượng mã “nóng” lớn ngay sát pipeline. Trong server CPU, lợi ích của lựa chọn này là rất thực dụng: các workload như Java runtime, database executor, RPC framework, service mesh, network stack hay thậm chí mã kernel nóng đều có thể đồng thời chiếm một vùng footprint lệnh đáng kể. I-cache lớn kết hợp với ITLB đủ dày giúp giảm đáng kể áp lực do instruction miss và translation miss, đặc biệt trong môi trường có nhiều process, nhiều thread và workload hợp nhất từ nhiều lớp phần mềm khác nhau.

Khối giải mã của N Core cũng được tổ chức theo hướng ưu tiên bằng thông frontend. Sơ đồ cho thấy sự hiện diện của **uCode, Op-Cache** và **6 simple decoder**, sau đó đi vào **Decode Queue** với thông lượng biểu diễn là **6 MOPs**. Đây là một dấu hiệu rất rõ rằng kiến trúc không chỉ trông chờ vào decode trực tiếp từng chu kỳ, mà còn sử dụng **op-cache** để tái sử dụng các chuỗi thao tác đã được giải mã sẵn. Trong thực tế server, những đoạn mã lặp lại

rất nhiều: vòng lặp xử lý request, packet processing loop, parser loop, transaction logic, runtime dispatch, interpreter hot path, TLS record handling, compression path. Một op-cache tốt sẽ giảm áp lực lên decoder, giảm điện năng ở frontend, đồng thời giúp giữ thông lượng ổn định khi chạy các path “nóng” lặp đi lặp lại hàng tỷ lần. Việc có thêm **uCode** cũng hàm ý rằng N Core vẫn sẵn sàng xử lý các lệnh phức hợp hoặc các trường hợp đặc biệt một cách linh hoạt, phù hợp với yêu cầu đầy đủ tính năng của một lõi server-class.

Ngay phía sau decode là phần **rename, quản lý thanh ghi vật lý và retirement**, nơi bộc lộ rất rõ tham vọng của kiến trúc. Sơ đồ thể hiện **~250 INT/FP Unified PRF, 360 Physical Register Reclaim Table (PRT)** và **320 Coalesced Retire Queue (ROB)**. Dù cách đặt tên nội bộ có thể khác nhau tùy tổ chức thiết kế, ý nghĩa kiến trúc là rất rõ: đây là một lõi có **cửa sổ ngoài thứ tự sâu**, đủ để giữ một lượng lớn micro-ops đang bay trong pipeline, qua đó hấp thụ tốt độ trễ từ nhiều nguồn khác nhau như cache miss, nhánh khó, phép chia, truy cập phụ thuộc và các chuỗi tính toán dài. Với **server workloads**, khả năng này thường quan trọng hơn nhiều so với việc chỉ tăng xung hay tăng số ALU thuần túy. Phần lớn ứng dụng backend không vector hóa hoàn hảo; hiệu năng của chúng đến từ khả năng của lõi trong việc liên tục tìm ra instruction-level parallelism giữa các chuỗi lệnh đan xen, giữ pipeline bận rộn trong khi chờ dữ liệu, và phục hồi nhanh sau các pha hụt song song tạm thời. Một **ROB khoảng 320 entry** đặt N Core vào nhóm thiết kế rất nghiêm túc, đủ để xem là một lõi máy chủ cao cấp chứ không còn là lõi client thông thường phóng lớn.

Khối **issue** và các scheduler tiếp tục củng cố nhận định đó. Sơ đồ cho thấy N Core không dùng một hàng chờ phát lệnh chung đơn giản, mà được chia thành nhiều cụm **scheduler chuyên trách**, bao gồm các cụm **32 FP Scheduler, 32 None Scheduler**, nhiều cụm **20 INT Scheduler** và **24 INT Scheduler**, cùng **32 MEM Scheduler**. Sự phân tách này thường phản ánh triết lý cân bằng giữa độ rộng, độ trễ, chi phí wakeup/select và độ phức tạp dây dẫn nội khối. Nói cách khác, N Core được thiết kế để vừa **rộng** vừa **thực dụng**: mỗi lớp tác vụ có hàng đợi và vùng cấp phát tài nguyên riêng, giảm tắc nghẽn chéo giữa nhánh, số nguyên, dấu chấm động và bộ nhớ. Với workload server đa dạng, đây là một lợi thế lớn, bởi luồng thực thi hiếm khi thuần một loại tác vụ; một request động chạm đồng thời đến parsing, pointer chasing, integer arithmetic, crypto, memory load/store và đôi khi cả SIMD.

Ở phía **integer backend**, N Core có các đơn vị **Branch, ALU, ALU/MUL, ALU/MUL/DIV** được phân bổ qua nhiều cụm scheduler khác nhau. Cấu hình như vậy cho thấy đây là một lõi được tối ưu rất rõ cho mã backend “đời thật”: nhiều phép toán địa chỉ, nhiều thao tác logic, nhiều nhánh, nhiều vòng lặp, xen kẽ với các phép nhân/chia không thường xuyên

nhưng vẫn cần hiệu năng đủ tốt. Trong một **ARM server CPU**, phần integer backend này chính là nơi gánh phần lớn công việc của hệ điều hành, scheduler, network stack, transaction processing, language runtime, key-value lookups, metadata management, storage logic, logging, access control và tất cả các loại “business logic” mà trung tâm dữ liệu phải xử lý mỗi giây. N Core vì thế không đi theo triết lý thiên lệch cực đoan về SIMD hay AI, mà vẫn duy trì một nền integer rất dày và thực dụng — một đặc trưng quan trọng của lõi server đúng nghĩa.

Tuy nhiên, N Core không dừng lại ở integer. Phía **floating-point và vector** của lõi này cũng được đầu tư đủ mạnh để không trở thành điểm nghẽn khi workload mở rộng sang các miền dữ liệu hiện đại hơn. Sơ đồ cho thấy nhiều cụm thực thi **VALU / VMUL / FP ADD**, kèm theo các đơn vị riêng như **FP MUL** và **FP DIV**. Điều này gợi ý rằng N Core được thiết kế để hỗ trợ tốt các phép toán SIMD và dấu chấm động ở mức đủ mạnh cho các tác vụ như xử lý multimedia, phân tích dữ liệu, nén/giải nén, khoa học tính toán mức vừa, inference nhẹ, xử lý tín hiệu, và tăng tốc các thư viện chuẩn sử dụng vector. Trong bối cảnh máy chủ ARM ngày nay, đây là một lựa chọn rất hợp thời: server không còn chỉ chạy web request hay database thuần túy, mà ngày càng thường xuyên phải xử lý **TLS, compression, analytics, AI inference nhỏ, telemetry processing, recommendation filtering**, hay các pipeline ETL nhẹ trực tiếp trên CPU. Một lõi server mạnh vì thế cần vector đủ tốt để gánh các workload lại này mà không phải lệ thuộc hoàn toàn vào accelerator ngoài.

Sự xuất hiện của một khối **AES** chuyên dụng là một chi tiết đặc biệt đáng giá. Trong môi trường datacenter hiện đại, **mã hóa** không còn là tác vụ ngoại lệ; nó là mặc định. Kết nối giữa client và server gần như luôn dùng TLS, traffic nội bộ giữa các service ngày càng nhiều nơi cũng mã hóa, dữ liệu lưu trữ thường được bảo vệ bằng encryption at rest, còn nhiều ứng dụng doanh nghiệp có thêm yêu cầu ký số, xác thực và bảo mật đầu cuối. Khi có một đơn vị **AES acceleration** riêng, N Core có thể giảm mạnh overhead bảo mật trên mỗi request, nâng throughput HTTPS, giảm tiêu hao điện cho cùng một lượng lưu lượng, và cải thiện độ ổn định của latency dưới tải lớn. Đây là chi tiết rất “server-minded”: không hào nhoáng như đơn vị tensor, nhưng mang giá trị cực kỳ lớn ở triển khai thực tế.

Nếu frontend thể hiện tham vọng về control flow, còn backend thể hiện độ rộng và tính đa dụng, thì **memory subsystem** của N Core mới là nơi bộc lộ rõ nhất bản chất server-class của thiết kế. Sơ đồ cho thấy **32 MEM Scheduler**, các khối **Store Data, AGU, AGU, Load Data, Load AGU**, cùng với **84 Store Buffer** và **96 Load Buffer**. Đây là cấu hình rất mạnh cho một core tổng dụng, và nó gửi đi một thông điệp rất rõ: N Core được thiết kế để sống

khỏe trong những môi trường mà độ trễ bộ nhớ và tính không đều của truy cập dữ liệu là vấn đề trung tâm. Đó chính là đặc trưng của rất nhiều workload máy chủ: cơ sở dữ liệu với chỉ mục nhiều tầng, key-value store, đồ thị, search engine, object store metadata, packet processing, distributed runtime, cache phân tán, code nhiều con trỏ và nhiều truy cập gián tiếp. Các ứng dụng như vậy thường không bị giới hạn chủ yếu bởi FLOPS, mà bởi việc lỗi có thể duy trì bao nhiêu load/store đang chờ, có thể theo dõi bao nhiêu miss độc lập, và có thể tận dụng tốt thế nào các khoảng trống khi đang chờ dữ liệu quay về. Với **load/store buffer** cỡ lớn, N Core có năng lực rất tốt trong việc che giấu memory latency, duy trì MLP (*memory-level parallelism*) và tránh để pipeline bị “đói” khi dữ liệu về chậm.

Khối cache dữ liệu và TLB cũng được xây rất bài bản. Ở mức gần lõi có **L1 DTLB 192 entry** và **64KB L1 D-Cache, 8-way**, còn phía dưới là **L2 TLB 2048 entry** và **3MB L2 Cache, 16-way**. Cấu hình này cho thấy N Core đặc biệt coi trọng hai điểm: **dịch địa chỉ hiệu quả** và **working set dữ liệu trên mỗi core đủ lớn**. Trong server CPU, đây là hai yếu tố cốt lõi. Các hệ thống chạy container, VM, database, microservice và nhiều process thường có footprint dịch địa chỉ rất rộng; TLB nông sẽ làm hiệu năng rơi rất nhanh khi bước ra khỏi vùng làm việc nhỏ. Việc có **L2 TLB 2048 entry** cho thấy thiết kế chuẩn bị sẵn cho những áp lực đó. Còn **L2 riêng 3MB, 16-way** là một con số đáng chú ý: nó hàm ý rằng mỗi core có khả năng giữ lại lượng dữ liệu nóng tương đối lớn trước khi phải chạm tới LLC hay DRAM, giảm áp lực liên core, giảm xung đột trên tài nguyên chia sẻ và đặc biệt hữu ích cho workload có working set trung bình nhưng nhạy với latency. Đây là một lựa chọn rất đúng tinh thần server, nơi **hiệu năng bền vững** và **độ ổn định của tail latency** thường đáng giá không kém peak throughput.

Các đường băng thông nội bộ được đánh dấu **32 B/cycle** giữa các tầng cache và khối load/store cho thấy nhà thiết kế đã chú ý đến **sự cân đối trong dòng dữ liệu**, chứ không chỉ tăng số đơn vị thực thi trên giấy. Một lõi server tốt không thể chỉ rộng ở issue rồi để nghẽn ở cache hay load pipe; nó phải có khả năng nuôi backend ổn định. Từ những gì sơ đồ thể hiện, N Core là một thiết kế theo hướng cân chỉnh như vậy: frontend đủ khỏe để không hụt lệnh, backend đủ rộng để tiêu hóa nhiều loại tác vụ, còn memory subsystem đủ dày để không sụp ngay khi workload bước sang miền memory-bound.

Khối **retirement** với thông lượng **6 μ ops** cũng là một chi tiết đáng bàn. Nó cho thấy N Core theo đuổi cách tiếp cận cân bằng: lõi có thể rất rộng và sâu ở bên trong, nhưng khâu commit vẫn giữ ở mức hợp lý để đảm bảo tính khả thi về điện năng, độ ổn định thời gian và chi phí thiết kế. Với CPU máy chủ, đây là một lựa chọn rất thường gặp và rất đúng đắn. Datacenter

không chỉ cần một lõi chạy nhanh ở benchmark ngắn; họ cần một lõi duy trì hiệu năng cao hàng giờ, hàng ngày, trong điều kiện nhiệt, điện và tải thay đổi liên tục. Vì vậy, những kiến trúc thành công thường là những kiến trúc biết **cân bằng giữa độ rộng, độ sâu và hiệu quả**, thay vì đẩy một chỉ số nào đó lên cực hạn.

Nếu đặt N Core vào vai trò hệ thống, có thể nói đây là một **lõi ứng dụng chính** của một **CPU ARM server hiện đại**. Nó đủ mạnh để trực tiếp chạy **kernel, hypervisor, container orchestration, database, web tier, middleware, microservices, storage service, network function**, và nhiều loại workload hỗn hợp khác. Khác với một mô hình host–accelerator, nơi có một lõi điều phối tổng quát bên cạnh các engine chuyên biệt, N Core ở đây mang dáng dấp của lõi “đa năng chủ lực”: chính bản thân nó đã có predictor mạnh, frontend rộng, cửa sổ OoO sâu, integer backend lớn, FP/vector đủ mạnh, crypto acceleration, và hệ thống nhớ đậm chất server. Nói ngắn gọn, đây là kiểu lõi được sinh ra để **làm việc chính** trong socket máy chủ.

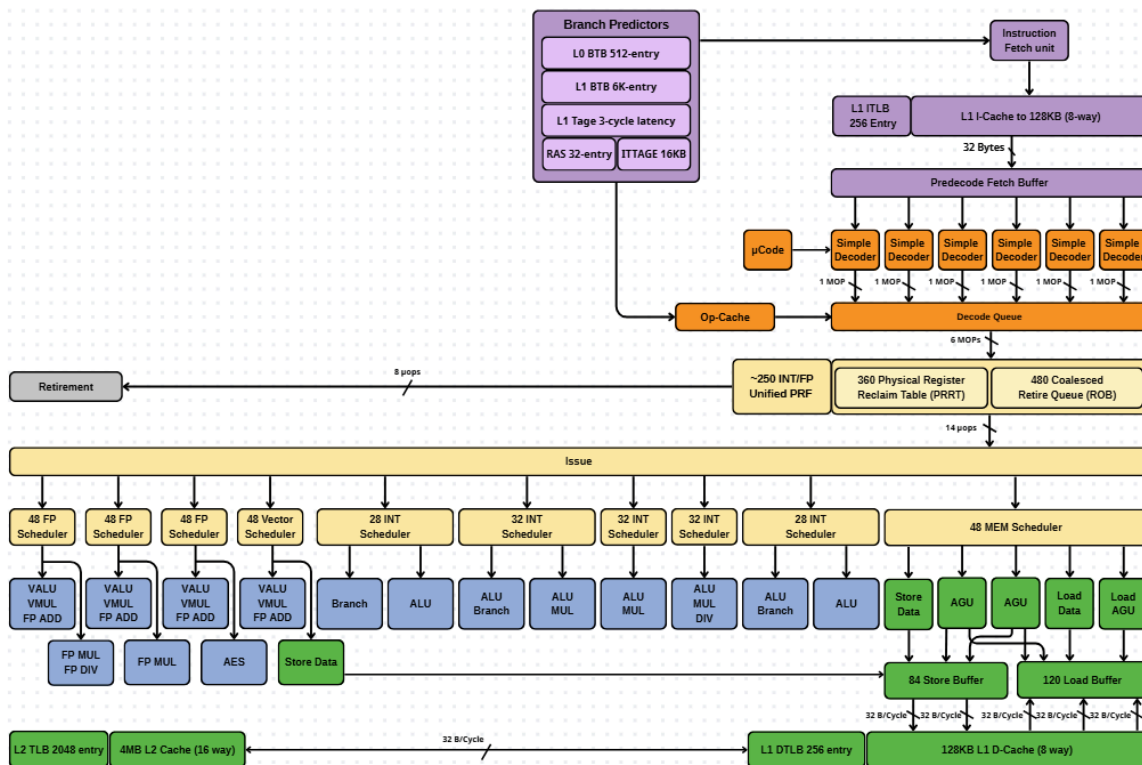
Về mặt triết lý thiết kế, điều đẹp ở N Core là nó không quá thiên lệch vào một cực nào. Nó không chỉ là một lõi “control-heavy” đơn thuần, vì vector/FP và AES đủ mạnh để gánh tốt nhiều tác vụ dữ liệu hiện đại. Nhưng nó cũng không phải một compute core thiên SIMD đến mức bỏ quên code rẽ nhánh phức tạp, bởi predictor, decode, ROB và integer backend đều rất chắc. Nó cũng không chạy theo lối nghĩ “nhiều execution unit là đủ”, vì memory subsystem, TLB và cache đều được đầu tư đúng mức. Chính **sự cân bằng này** mới là điều khiến N Core phù hợp với server ARM: workload máy chủ trong thực tế luôn lẫn lộn giữa control, data, crypto, memory latency và concurrency, nên một kiến trúc tốt phải xử lý được tất cả ở mức cao, thay vì chỉ cực mạnh ở một chiều.

Nếu cần chốt lại bằng một đoạn mang tính tài liệu kỹ thuật, có thể viết như sau:

N Core là một lõi ARM server-class hiệu năng cao, theo kiến trúc out-of-order, độ rộng lớn, được tối ưu cho các workload máy chủ hiện đại có đặc trưng control flow phức tạp, mức độ song song lệnh vừa đến cao, áp lực bộ nhớ lớn và yêu cầu bảo mật tích hợp. Với frontend dự đoán nhánh mạnh, instruction supply băng thông cao, cửa sổ đổi tên và ngoài thứ tự sâu, backend thực thi cân bằng giữa integer, vector/floating-point và crypto, cùng hệ thống load/store, TLB và cache dày đặc, N Core phù hợp để đóng vai trò lõi xử lý chủ lực trong các CPU ARM servers dùng cho cloud, virtualization, database, mạng, lưu trữ và dịch vụ backend quy mô lớn.

N core Premium

N Core Premium



N Core Premium có thể được mô tả như một lõi **ARM server-class** hiệu năng cao theo triết lý **out-of-order, wide-issue, high-throughput**, được thiết kế để đảm nhiệm trực tiếp các workload trung tâm của hệ thống máy chủ hiện đại. Nếu N Core đã cho thấy đáng đáp của một lõi tổng dụng mạnh, thì **N Core Premium** là phiên bản được đẩy lên một bậc rõ rệt ở gần như mọi mặt quan trọng: năng lực dự đoán nhánh, khả năng nuôi lệnh ở frontend, độ sâu của cửa sổ thực thi ngoài thứ tự, bề rộng của cụm scheduler, năng lực vector và số nguyên, cũng như quy mô của hệ thống nhớ gần lõi. Nhìn tổng thể, đây không phải là một thiết kế tối ưu cho một kiểu tác vụ hẹp, mà là một kiến trúc được cân chỉnh để duy trì hiệu năng cao, ổn định và bền vững trên các tải máy chủ phức tạp, nơi control flow, memory latency, crypto và data processing thường đan xen trong cùng một luồng xử lý.

Đi từ phía **frontend**, N Core Premium cho thấy một quan điểm thiết kế rất rõ: muốn hiệu năng backend thực sự phát huy, lõi phải được “nuôi” bằng luồng lệnh dồi dào và chính xác ngay từ đầu. Cụm **Branch Predictors** gồm **L0 BTB 512-entry**, **L1 BTB 6K-entry**, **L1 TAGE 3-cycle latency**, **RAS 32-entry** và **ITTAGE 16KB** cho thấy lõi này được đầu tư bài bản cho bài toán điều hướng luồng lệnh. Đây là một đặc điểm rất quan trọng trong môi trường server, bởi những workload thực tế như kernel, hypervisor, database engine, middleware,

network stack hay runtime của ngôn ngữ bậc cao thường có mật độ nhánh lớn, nhiều lời gọi hàm, nhiều nhánh gián tiếp và các mẫu hành vi không dễ dự đoán. Một bộ dự đoán nhánh mạnh không chỉ nâng IPC trung bình, mà còn làm giảm các cú hực nhịp lớn trong pipeline, từ đó cải thiện độ ổn định của throughput và hạn chế những đuôi trễ không mong muốn trong các hệ thống đòi hỏi phản hồi đều.

Ngay sau đó, nhánh nạp lệnh tiếp tục củng cố hình ảnh của một lõi server cao cấp. Sơ đồ cho thấy **Instruction Fetch Unit** được hậu thuẫn bởi **L1 ITLB 256 entry** và **L1 I-Cache lên tới 128KB, 8-way**, với đường cấp dữ liệu **32 Bytes** trước khi đi vào **Predecode Fetch Buffer**. Chỉ riêng việc nâng **ITLB từ mức thường thấy lên 256 entry** cũng đã nói lên khá nhiều: N Core Premium được chuẩn bị để vận hành tốt trong môi trường có áp lực dịch địa chỉ lệnh lớn, nhiều tiến trình, nhiều context và footprint mã rộng. Trong khi đó, **I-Cache 128KB** là một dung lượng đáng kể cho một lõi tổng dụng, cho phép giữ nhiều vùng mã nóng ở gần pipeline hơn, giảm xác suất instruction miss trên các đường thực thi quan trọng. Trong thực tế máy chủ, điều này đặc biệt hữu ích khi lõi phải liên tục chuyển qua lại giữa các lớp phần mềm khác nhau như kernel, runtime, framework và ứng dụng.

Khối giải mã của N Core Premium cũng cho thấy một frontend được tổ chức để ưu tiên băng thông và hiệu quả năng lượng. Sự hiện diện của **uCode**, **Op-Cache** và **6 simple decoder** cho thấy kiến trúc không chỉ dựa vào decode trực tiếp theo kiểu truyền thống, mà còn tận dụng **op-cache** để tái sử dụng các chuỗi thao tác đã giải mã sẵn. Cách tổ chức này đặc biệt phù hợp với các workload server có nhiều hot path lặp đi lặp lại như xử lý request, packet loop, parsing, dispatch trong runtime hay các nhánh logic ổn định ở tầng middleware. Khi những chuỗi lệnh này được lấy trực tiếp từ op-cache, áp lực lên decode sẽ giảm, độ trễ frontend được cải thiện và hiệu năng trên watt cũng tốt hơn. **Decode Queue 6 MOPs** cho thấy lõi vẫn duy trì một băng giải mã cao, đủ để phục vụ cho một backend rất rộng ở phía sau.

Phần nổi bật nhất của N Core Premium nằm ở khu vực **đổi tên thanh ghi, quản lý tài nguyên đang bay và hàng đợi nghỉ hưu**, bởi đây là nơi bộc lộ rõ độ “sâu” của kiến trúc. Sơ đồ thể hiện **~250 INT/FP Unified PRF**, **360 Physical Register Reclaim Table (PRRT)** và đặc biệt là **480 Coalesced Retire Queue (ROB)**. Nếu coi ROB là thước đo trực quan cho mức độ tham vọng của một lõi out-of-order, thì con số **480** đặt N Core Premium vào nhóm thiết kế rất mạnh tay. Một ROB sâu như vậy cho phép lõi giữ số lượng lớn micro-ops đang thực thi đồng thời, hấp thụ tốt hơn độ trễ đến từ cache miss, branch miss, phép toán độ trễ cao và các chuỗi phụ thuộc dài. Với workload server vốn thường không “sạch” như

benchmark tính toán, đây là lợi thế đặc biệt đáng giá: lõi có thêm không gian để tìm instruction-level parallelism, sắp xếp lại công việc thông minh hơn và tiếp tục đẩy những lệnh độc lập đi trước trong khi chờ dữ liệu của các lệnh khó quay về.

Từ khối rename xuống **Issue**, sơ đồ cho thấy **14 μops** được đưa vào backend, trong khi nhánh **Retirement** thể hiện mức **8 μops**. Điều này gợi ra một cấu trúc cân bằng rất điển hình của lõi server cao cấp: bên trong, kiến trúc đủ rộng và đủ sâu để tích lũy nhiều công việc, nhưng khâu commit vẫn được giữ ở một mức hợp lý để đảm bảo kiểm soát tốt điện năng, timing và độ ổn định lâu dài. Với CPU máy chủ, đây thường là lựa chọn đúng đắn hơn việc chạy theo một con số cực hạn ở một khối đơn lẻ. Điều quan trọng không phải là một vài chu kỳ “đẹp” trên giấy, mà là khả năng duy trì throughput cao và bền vững trong thời gian dài, dưới tải thực và trong điều kiện nhiệt, điện phức tạp của datacenter.

Bước sang phía **scheduler và execution backend**, N Core Premium cho thấy quy mô tài nguyên đã được nâng lên rõ rệt so với một lõi tổng dụng thông thường. Ở phía vector và dấu chấm động, lõi có ba cụm **48 FP Scheduler** và một cụm **48 Vector Scheduler**, nối tới bốn khối thực thi kiểu **VALU / VMUL / FP ADD**. Việc phân bổ như vậy cho thấy N Core Premium không chỉ mạnh ở integer control code, mà còn có khả năng xử lý rất tốt các tác vụ vector hóa và floating-point ở mật độ cao. Đây là chi tiết đáng chú ý, bởi server hiện đại ngày càng phải xử lý các tác vụ lai như nén/giải nén, analytics, xử lý dữ liệu số, inference nhẹ, xử lý media nội bộ, các thư viện toán học tối ưu hóa và các phần của pipeline AI chạy trên CPU. Một lõi server cao cấp vì thế không thể chỉ mạnh ở branch và ALU; nó cần một nền vector đủ dày để không trở thành nút thắt khi workload thay đổi.

Bên dưới các cụm thực thi đó, sơ đồ còn cho thấy các đơn vị chuyên biệt như **FP MUL / FP DIV**, **FP MUL** riêng và **AES**. Cấu trúc này nói lên hai điều. Thứ nhất, N Core Premium dành tài nguyên thực cho những phép toán có độ trễ cao hoặc nhu cầu riêng biệt, thay vì dồn tất cả vào vài pipeline đa năng. Thứ hai, kiến trúc đặc biệt chú trọng các workload có yếu tố **mã hóa**. Sự hiện diện của khối **AES** là một tín hiệu rất rõ về định hướng server: trong môi trường datacenter, encryption không còn là phụ trợ mà đã trở thành tải nền. HTTPS/TLS, lưu trữ mã hóa, bảo mật dịch vụ nội bộ và nhiều cơ chế xác thực hiện đại đều liên tục tiêu thụ tài nguyên mã hóa. Một đơn vị AES phần cứng giúp giảm đáng kể overhead bảo mật, nhờ đó tăng throughput tổng thể và giảm chi phí điện năng cho cùng một khối lượng công việc.

Ở phía **integer backend**, N Core Premium được trang bị các cụm **28 INT Scheduler, 32 INT Scheduler, 32 INT Scheduler, 32 INT Scheduler** và một cụm **28 INT Scheduler** nữa, cho thấy tài nguyên số nguyên được trải rộng và phân vùng tương đối kỹ. Bên dưới là các đơn vị **Branch, ALU, ALU Branch, ALU MUL, ALU MUL, ALU MUL DIV, ALU Branch** và **ALU**. Cách tổ chức này đặc biệt phù hợp với các workload thực tế của máy chủ, nơi luồng lệnh thường là sự pha trộn liên tục giữa thao tác địa chỉ, logic điều khiển, so sánh nhánh, số học nguyên, phép nhân/chia và nhiều chuỗi phụ thuộc ngắn. Thay vì thiết kế backend theo kiểu thiên lệch quá nhiều sang một loại tác vụ, N Core Premium cho thấy một tư duy rất thực dụng: duy trì nền integer dày, đủ số cổng xử lý và đủ bề rộng scheduler để đảm bảo những công việc “ít hào nhoáng nhưng sống còn” của phần mềm server luôn được xử lý thông suốt.

Một trong những khối nói lên nhiều nhất về đẳng cấp server của N Core Premium là **memory subsystem**. Lõi này có **48 MEM Scheduler**, đi kèm các khối **Store Data, AGU, AGU, Load Data** và **Load AGU**. Ở tầng buffer, sơ đồ cho thấy **84 Store Buffer** và **120 Load Buffer**. Đây là những con số rất đáng kể và mang ý nghĩa kiến trúc rất rõ: N Core Premium được thiết kế để sống khỏe trong những môi trường mà memory-level parallelism là yếu tố quyết định. Những workload như cơ sở dữ liệu, key-value store, graph processing, packet handling, distributed cache, storage metadata hay runtime nhiều con trở thường bị giới hạn bởi độ trễ bộ nhớ và tính rời rạc của truy cập dữ liệu. Khi lõi có thể theo dõi đồng thời nhiều load/store hơn, giữ được nhiều miss độc lập hơn và đẩy các truy cập bộ nhớ song song hiệu quả hơn, pipeline sẽ ít bị đói dữ liệu hơn và khả năng che giấu độ trễ DRAM sẽ tốt hơn đáng kể. **120 load buffer** là một nâng cấp rất giàu ý nghĩa, vì trong rất nhiều workload server, áp lực phía tải dữ liệu vào luôn nặng hơn phía ghi ra.

Tầng cache và TLB dữ liệu của N Core Premium tiếp tục thể hiện sự nâng cấp toàn diện. Ở gần lõi, ta thấy **L1 DTLB 256 entry** và đặc biệt là **128KB L1 D-Cache, 8-way**. So với mức 64KB thường gặp ở nhiều thiết kế tổng dụng, **L1 D-Cache 128KB** cho thấy N Core Premium ưu tiên rõ ràng cho việc giữ working set dữ liệu nóng ngay sát load/store engine, từ đó giảm độ trễ truy cập ở những đoạn mã nhạy cảm với latency. Phía dưới, sơ đồ ghi **L2 TLB 2048 entry** và **4MB L2 Cache, 16-way**. Đây là một cấu hình rất mạnh cho cache riêng cấp lõi. **L2 4MB** không chỉ tăng xác suất giữ được working set trung bình mà còn làm giảm áp lực lên các tài nguyên cache chia sẻ ở tầng cao hơn, từ đó cải thiện tính ổn định khi nhiều core cùng hoạt động. Với server CPU, lợi ích của một L2 riêng lớn không chỉ nằm ở throughput, mà còn ở việc giảm biến thiên độ trễ giữa các luồng công việc.

Các đường dữ liệu **32 B/cycle** giữa các tầng cache và khối xử lý bộ nhớ cho thấy N Core Premium không phải là một kiến trúc chỉ “phình to” về dung lượng, mà còn được cân đối về băng thông nội lõi. Đây là điểm rất quan trọng. Một thiết kế nhiều ALU hay scheduler nhưng không có đủ băng thông bộ nhớ gần lõi sẽ sớm bị nghẽn và không tận dụng được độ rộng danh nghĩa. Trong sơ đồ này, có thể thấy N Core Premium được xây theo tư duy đồng bộ hơn: frontend mạnh, rename sâu, backend rộng và bộ nhớ gần lõi đủ dày để cấp dữ liệu cho khối thực thi.

Nếu nhìn N Core Premium như một thực thể hoàn chỉnh, điều nổi bật nhất không chỉ là từng con số riêng lẻ, mà là **sự đồng bộ trong triết lý mở rộng**. Frontend được tăng cường để giảm mất mát do branch và fetch. Cửa sổ ngoài thứ tự được làm sâu để hấp thụ latency. Scheduler được mở rộng để nâng năng lực chọn lệnh song song. Integer, vector, FP và AES cùng được đầu tư để tránh lệch tải. Hệ thống load/store, cache và TLB được nới rộng để backend không bị bỏ đói. Đây chính là dấu hiệu của một **premium core** đúng nghĩa: không phải một lõi chỉ được cộng thêm vài đơn vị thực thi, mà là một kiến trúc được nâng cấp theo chiều sâu, nơi mọi tầng của pipeline đều được đẩy lên để hỗ trợ lẫn nhau.

Nếu cần viết theo văn phong tài liệu kỹ thuật trau chuốt, có thể dùng đoạn sau:

N Core Premium là lõi ARM server-class hiệu năng cao, theo kiến trúc out-of-order và wide-issue, được thiết kế để xử lý trực tiếp các workload máy chủ hiện đại với yêu cầu cao về thông lượng, độ ổn định và khả năng che giấu độ trễ. Kiến trúc này sở hữu frontend mạnh với hệ dự đoán nhánh nhiều tầng, L1 ITLB 256 entry, L1 I-Cache 128KB và cơ chế op-cache nhằm đảm bảo nguồn cấp lệnh băng thông cao cho backend. Ở trung tâm lõi, cửa sổ thực thi ngoài thứ tự được mở rộng với khoảng 250 thanh ghi vật lý hợp nhất cho INT/FP, bảng quản lý thu hồi thanh ghi 360 entry và ROB hợp nhất tới 480 entry, cho phép duy trì lượng lớn micro-ops đang bay để khai thác tối đa song song mức lệnh và hấp thụ hiệu quả độ trễ từ bộ nhớ cũng như các phép toán phức tạp. Phía backend, N Core Premium cung cấp cụm scheduler rộng cho cả integer, floating-point, vector và memory, cùng nhiều đơn vị thực thi chuyên biệt như ALU, branch, multiply/divide, VALU, VMUL, FP add, FP mul/div và AES acceleration. Cách tổ chức này cho phép lõi xử lý hiệu quả đồng thời các workload điều khiển phức tạp, tính toán số nguyên, SIMD/floating-point và mã hóa bảo mật. Về hệ thống nhớ, lõi được trang bị 48 memory scheduler, 84 store buffer, 120 load buffer, L1 DTLB 256 entry, L1 D-Cache 128KB và L2 Cache riêng 4MB, tạo nên một nền tảng rất mạnh cho các ứng dụng memory-intensive đặc trưng của môi trường server. Tổng thể, N

Core Premium thể hiện rõ vai trò của một lõi xử lý chủ lực trong CPU ARM máy chủ cao cấp, nơi hiệu năng không chỉ đến từ độ rộng tính toán, mà từ sự cân bằng chặt chẽ giữa dự đoán nhánh, cấp lệnh, thực thi ngoài thứ tự và năng lực dữ liệu gần lõi.